
A Post-Quantum Aware Security Credential Management System for V2X Communications Using Selective Cryptographic Methodology

Students

Ahmed Fikry	320220026
Ebrahim Ramadan	320220029
Mazen Mohamed	320220031
Mostafa Atef	320220034
Kareem Kamel	320220046
Ahmed Saber	320220047

Supervisors

Prof. Samir El-Sagheer

*A project report submitted in partial fulfillment of the requirements
for B.Sc. degree in Computer Science and Information Technology*

Egypt-Japan University of Science and Technology (E-JUST)
Final Year 2025/2026

Table of Contents

Page	
List of Figures.....	iii
List of Tables.....	iv
Certificate.....	v
Dedication.....	vi
Acknowledgement.....	vii
Abstract.....	viii
Chapter 1: Introduction	1
Chapter 2: System Overview.....	15
Chapter 3: Modeling and System Design.....	30
Chapter 4: Implementation Process and Testing.....	50
Chapter 5: Discussion of the Results.....	75
Chapter 6: AI-Driven Intrusion Detection System for V2X Communications.....	69
References	85
Appendices	90

List of Figures

- Fig. 1: V2X Communication Ecosystem Architecture
- Fig. 2: Security Credential Management System (SCMS) Architecture (with PQC Key Management)
- Fig. 3: Vehicle-to-Vehicle (V2V) Communication Flow
- Fig. 4: Vehicle-to-Infrastructure (V2I) Message Exchange
- Fig. 5: Certificate Lifecycle Management Process (Hybrid Signature Support)
- Fig. 6: System Component Diagram
- Fig. 7: Message Authentication and Verification Flow (Dual Signature Verification)
- Fig. 8: Dashboard Interface for Real-time Monitoring
- Fig. 9: Docker Container Architecture
- Fig. 10: Certificate Issuance and Revocation Workflow (Classical and PQC Keys)
- Fig. 11: System Performance Metrics Under Normal Load
- Fig. 12: System Performance Under Simulated Attack Scenarios
- Fig. 13: Project Timeline - Gantt Chart
- Fig. 14: Vehicle Simulation State Machine
- Fig. 15: Database Schema for Certificate Management (PQC Fields Included)
- Fig. 16: Hybrid Signature Scheme Architecture (RSA + Dilithium)
- Fig. 17: Post-Quantum Key Generation Process
- Fig. 18: Performance Comparison: Classical vs. PQC Signatures

List of Tables

Table 1: Comparison of V2X Security Approaches (Classical vs. Post-Quantum)

Table 2: System Functional Requirements (Including PQC Capabilities)

Table 3: Hardware and Software Requirements (PQC Libraries Included)

Table 4: Component Cost Analysis

Table 5: Test Case Scenarios and Results (Including PQC Performance Tests)

Table 6: Performance Benchmarks Comparison (RSA vs. Dilithium vs. Hybrid)

Table 7: Development Timeline - Actual vs. Estimated

Table 8: Attack Detection Success Rates

Table 9: Post-Quantum Cryptography Algorithm Comparison

Table 10: Signature Size and Performance Metrics (Classical vs. PQC)

Certificate

I certify that the project report has been prepared and written under my direct supervision and guidance.

This report is approved for submission for its evaluation.

Prof. Supervisor Samir El-Sagheer

Dedication

I dedicate this work to my family and friends who have supported me throughout this journey. I would like to thank everyone who believed in this project and contributed to its success through their guidance, encouragement, and technical expertise.

Ahmed Fikry, Ebrahim Diab, Mazen Mohamed, Mostafa Atef, Kareem Kamel, Ahmed Saber

Acknowledgement

I would like to express my deepest gratitude to:

- My supervisors, Prof. Samir El-Sagheer, for their invaluable guidance, continuous support, and expert advice throughout this project.
- The faculty and staff of the Computer Science and Information Technology department at E-JUST for providing the resources and environment necessary for this research.
- My colleagues and peers who participated in testing, provided feedback, and helped troubleshoot technical challenges during development.
- The College of Computing for providing access to research materials, computing resources, and development tools.
- My family for their unwavering support, patience, and encouragement throughout my academic journey.
- The open-source community whose libraries, frameworks, and documentation made this implementation possible.

Ahmed Fikry, Mostafa Atef, Ebrahim Diab, Mazen Mohamed, Kareem Kamel, Ahmed Saber

Abstract

Vehicle-to-everything (V2X) communication technology is an important development in intelligent transportation systems, enabling vehicles to communicate with each other, with infrastructure, and with vulnerable road users, in order to improve road safety and traffic efficiency. However, the wireless nature of the V2X communications poses several important security challenges such as message tampering, replay attacks, Sybil attacks and privacy. The project describes a Security Credential Management System (SCMS) for V2X communications that addresses these problems using cryptographic authentication, certificate management, certificate revocation, and real-time monitoring.

The proposed system uses principles of Public Key Infrastructure (PKI) tailored for vehicular networks to provide secure communication channels between vehicles and roadside infrastructure. It employs IEEE 1609.2-based digital signatures and a selective cryptographic approach in which high-frequency Cooperative Awareness Messages (CAMs) are secured using ECDSA-P256-SHA256, while safety-critical Decentralized Environmental Notification Messages (DENMs) are protected using the post-quantum CRYSTALS-Dilithium2 signature algorithm. This approach provides a trade-off between quantum-resistant security and real-time requirements of V2X communications.

The system was implemented using Python, Flask, Docker and modern cryptographic libraries. Experimental results demonstrate successful certificate issuance, message authentication, certificate verification, and secure message broadcasting while maintaining acceptable performance for real-time vehicular communication. This work contributes to the development of secure and quantum-aware vehicular communication systems, and lays a foundation for the deployment of future intelligent transportation systems.

Chapter 1

Introduction

1.1 Introduction

The automotive industry is experiencing a transformation towards intelligent and connected transportation systems. Vehicle-to-Everything (V2X) communication technology enables vehicles to communicate with other vehicles (V2V), roadside infrastructure (V2I), pedestrians (V2P) and network services (V2N) to create a complete ecosystem for improved road safety, traffic efficiency and mobility services.

According to recent transportation statistics, traffic fatalities remain a critical public health crisis globally. V2X technology has the potential to significantly improve road safety by providing situational awareness beyond the line of sight of traditional sensors. However, the wireless nature of V2X communications introduces critical security challenges, including message tampering, replay attacks, privacy violations, and unauthorized message injection.

The Security Credential Management System (SCMS) is the preferred solution to these challenges to provide a Public Key Infrastructure (PKI) designed for V2X communications. It allows the creation and administration of digital certificates that provide secure and trustworthy communication of messages between vehicles and infrastructure.

This project implements a V2X security framework based on SCMS principles, vehicle simulation and real-time monitoring capabilities. The proposed system adopts selective cryptographic approach to address the emerging quantum threats, using ECDSA-P256-SHA256 for high frequency Cooperative Awareness Messages (CAMs) and CRYSTALS-Dilithium2 for safety-critical Decentralized Environmental Notification Messages (DENMs). This approach provides quantum-resistant security and addresses the real-time performance constraints of vehicular communications.

1.2 Problem Statement and Summary of Functional Requirements

Core Problem:

How to design and deploy a secure, privacy-preserving and quantum-aware authentication framework for V2X communications which can against common cyber-attacks while meeting the real-time performance requirements of safety-critical applications

Functional Requirements:

1. Certificate Management:

- Create and distribute digital certificates to infrastructure devices and vehicles.
- Manage certificate lifecycle, including issuance and revocation.
- Maintain a Certificate Revocation List (CRL) accessible to all network participants.
- Support pseudonymous certificates for privacy preservation.

2. Message Authentication:

- Sign outgoing V2X messages using appropriate cryptographic algorithms.
- Verify incoming message signatures before processing.
- Validate certificate authenticity and revocation status.
- Ensure message integrity and non-repudiation.
- Implement a selective cryptographic methodology using ECDSA-P256-SHA256 for Cooperative Awareness Messages (CAMs) and CRYSTALS-Dilithium2 for safety-critical Decentralized Environmental Notification Messages (DENMs).

3. Vehicle Simulation:

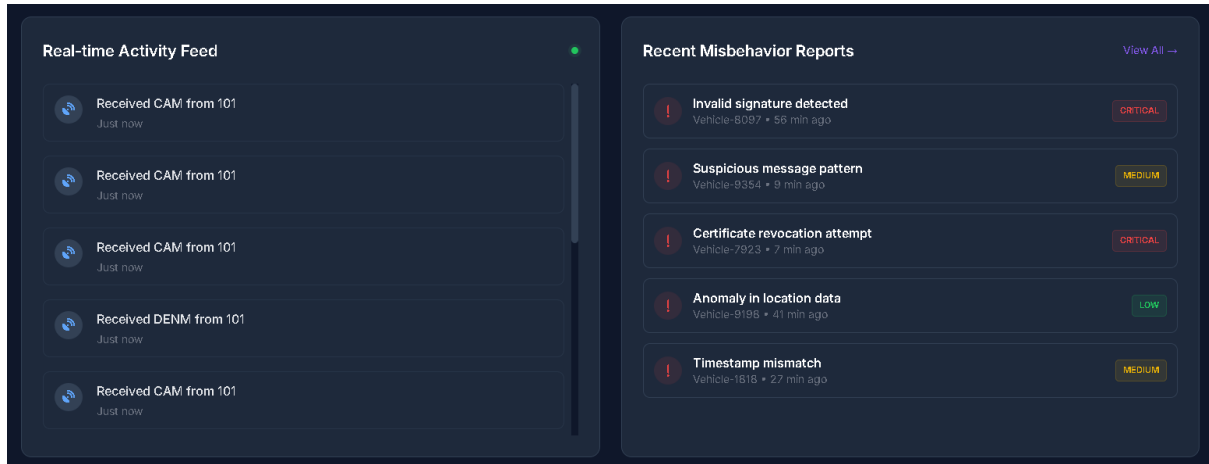
- Simulate multiple vehicles with unique identities and certificates.
- Generate realistic V2X messages, including CAM and DENM messages.
- Demonstrate vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communication.
- Support configurable vehicle profiles and movement parameters.

4. Infrastructure Management:

- Implement Roadside Unit (RSU) functionality.
- Broadcast infrastructure messages to vehicles.
- Receive and validate vehicle messages.
- Coordinate with SCMS services for certificate validation and management.

5. Security Monitoring:

- Provide real-time monitoring of system security status.
- Visualize message activity, certificate status, and revocation events.
- Maintain logging and auditing of security-relevant events.
- Monitor classical and post-quantum message distribution within the V2X network.



1.3 Goals & Objectives

1. Primary Goals:

- Implement SCMS core functionality with working certificate authority capabilities.
- Demonstrate secure V2X communications through vehicle and infrastructure simulation.
- Provide practical security validation against simulated attack scenarios.
- Create an accessible educational platform for V2X security learning.

2. Quantum-Resistant Security Foundation:

- Design a quantum-aware V2X security architecture based on a selective cryptographic methodology.
- Implement ECDSA-P256-SHA256 for high-frequency Cooperative Awareness Messages (CAMs).
- Implement CRYSTALS-Dilithium2 for safety-critical Decentralized Environmental Notification Messages (DENMs).
- Evaluate the performance implications of post-quantum cryptographic operations and their impact on real-time vehicular communications.

3. Specific Objectives:

- Design a modular architecture separating vehicles, infrastructure, SCMS, and dashboard components.
- Research and evaluate post-quantum cryptographic algorithms for V2X applications.
- Develop Python-based simulation modules for multiple vehicles and one RSU.
- Create a web-based dashboard for real-time monitoring.
- Implement certificate issuance, verification, and revocation mechanisms using SCMS principles.
- Implement a selective cryptographic methodology that employs ECDSA-P256-SHA256 for high-frequency CAM messages and CRYSTALS-Dilithium2 for safety-critical DENM messages.
- Measure the communication and computational overhead of post-quantum cryptographic operations.
- Successfully detect replay attacks, revoked certificate usage, and invalid signatures.
- Provide comprehensive testing coverage.
- Deploy the system using Docker containers.

1.4 Motivation

The motivation for this project stems from the growing need for secure and trustworthy vehicular communication infrastructure as connected and intelligent transportation systems continue to expand globally. The increasing adoption of V2X technologies requires robust security mechanisms that can protect communications against cyber-attacks while preserving the real-time performance requirements of safety-critical applications. Several factors motivated the development of this work:

1. • Educational Gap: Limited accessible implementations of V2X Security Credential Management Systems (SCMS) are available for educational and research purposes.
2. • Research Foundation: Providing a practical platform for future research in V2X security, certificate management, and quantum-aware vehicular communication systems.
3. • Real-World Impact: Contributing to transportation safety and reliability through secure, authenticated, and privacy-preserving vehicular communications.
4. • Technical Challenge: Addressing the complex integration of distributed systems, Public Key Infrastructure (PKI), cryptographic protocols, and real-time communication requirements.

5. • Industry Relevance: Developing practical knowledge and skills applicable to intelligent transportation systems, automotive cybersecurity, and next-generation connected vehicle technologies.

1.5 Overview of the Report

This report documents the complete development lifecycle:

Chapter 1: Introduction - Foundational context, problem definition, goals, motivation, and constraints

Chapter 2: System Overview - Literature review, system analysis, alternative designs, and proposed solution

Chapter 3: Modeling and System Design - Technical architecture, theoretical models, physical realization, and impact assessment

Chapter 4: Implementation and Testing - Development platform, code design, verification, validation, and economic analysis

Chapter 5: Discussion - Summary, critical appraisal, future enhancements, and sustainability

1.6 Primary Constraints

Technical Constraints:

- Real-time performance requirements (message processing < 100 ms).
- Cryptographic complexity and security correctness.
- Distributed system coordination and certificate management challenges.
- Integration of classical and post-quantum cryptographic algorithms while maintaining acceptable performance.

Resource Constraints:

- One semester development timeline (15 weeks).
- Standard computing hardware without actual V2X devices.
- Software dependency and container management.

Knowledge Constraints:

- Learning curve associated with V2X standards, SCMS architecture, and post-quantum cryptographic protocols.
- Limited implementation guidance and reference implementations for quantum-aware V2X security systems.

Scope Limitations:

- Educational and research platform rather than a production-ready system.
- Simulated V2X environment instead of real-world vehicular deployment.
- Focus on core SCMS functionality, certificate management, and selective cryptographic methodology.

1.8 Timeline

Phase I

Week 1-2: Literature review and project initialization.

Week 3-4: System requirements analysis and architecture design.

Week 5-6: SCMS core implementation and certificate management.

Week 7-8: Vehicle and infrastructure simulation development.

Week 9: Interim report preparation and system integration.

Week 10-11: Dashboard development and real-time monitoring implementation.

Week 12-13: Testing, validation, and performance evaluation.

Week 14: Documentation and final refinements.

Week 15: Final report preparation and project presentation.

Phase II

Week 1-2: Review of the first prototype and refinement of system requirements.

Week 3-4: Integration of selective cryptographic methodology and post-quantum security enhancements.

Week 5-6: Optimization of SCMS services, certificate management, and message authentication mechanisms.

Week 7-8: System integration, debugging, and deployment improvements.

Week 9-10: Comprehensive testing, security validation, and performance evaluation.

Week 11: Experimental investigation of an AI-based intrusion detection module as an independent bonus feature.

Week 12: Evaluation and analysis of the AI-based intrusion detection experiments.

Week 13-14: Final documentation, report writing, and paper preparation.

Week 15: Final presentation, demonstration, and project submission.

Chapter 2

System Overview

2.1 Literature Review

The design of secure Vehicle-to-Everything (V2X) communication systems has been a hot topic for research in the fields of vehicular networks, cryptographic security, and intelligent transportation systems. Modern V2X environments require strong authentication, privacy preservation, low latency and resistance against emerging cyber threats, while at the same time adhering to the strict real-time requirements of vehicular communications. Thus, Security Credential Management Systems (SCMS) have been widely adopted as the standard Public Key Infrastructure (PKI) framework to establish trust and secure message exchange in vehicular networks.

V2X Communication Standards:

The V2X security architectures are based on the IEEE 1609 family of standards, and in particular, IEEE 1609.2 specifies security services like certificate formats, digital signatures and message authentication mechanisms for vehicular communications. In contrast to traditional systems based on X.509 certificates, IEEE 1609.2 leverages lightweight certificate formats and pseudonymous identities to alleviate communication overheads and enhance vehicle privacy.

SAE International has also developed standards such as SAE J2735 and SAE J2945 that define V2X message formats and communication requirements for Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) applications. These standards define the format of Cooperative Awareness Messages (CAM) and Decentralized Environmental Notification Messages (DENM), which are some of the most important safety messages exchanged in V2X networks.

With the proliferation of V2X deployments, the security of these standards against classical and future quantum threats has become a major research direction. Thus, recent research has focused on Post-Quantum Aware Security Credential Management Systems (PQA-SCMS) and selective cryptographic approaches that balance quantum-resistant security with the low-latency requirements of vehicular communications.

Furthermore, recent works have studied Artificial Intelligence (AI) and Machine Learning (ML) techniques for V2X misbehavior/intrusion detection. AI based approaches can analyze vehicular communication patterns and identify anomalous behavior that can be representative of cyber-attacks or malicious nodes. In this project, the investigation of an AI-based intrusion detection model was an independent bonus research extension and is not part of the implemented SCMS architecture.

Post-Quantum Cryptography for V2X:

Quantum computing is a major threat to existing V2X security infrastructures based on classical public-key cryptographic algorithms such as ECDSA. It is well known that Shor's algorithm can efficiently break elliptic curve cryptography on a sufficiently powerful quantum computer, thus long-term V2X credentials are vulnerable to future attacks. To address this challenge, the National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardization project, and selected CRYSTALS-Dilithium (FIPS 204) as the standard digital signature algorithm, and CRYSTALS-Kyber (FIPS 203) as the standard key encapsulation mechanism. These algorithms offer good resistance against classical and quantum adversaries and have emerged as promising candidates for securing the next generation of V2X systems.

(Kampanakis et al. 2018) studies the applicability of post-quantum algorithms in vehicular networks and shows that lattice-based signatures like Dilithium can meet V2X performance demands with appropriate optimizations. But post-quantum signatures are much larger than ECDSA signatures (around 2-3 KB vs 64 bytes), which causes bandwidth problems for high-frequency V2X messaging.

Therefore, recent studies have investigated selective deployment strategies of post-quantum cryptography in V2X systems. Such approaches limit the use of post-quantum algorithms to long-term credentials and safety-critical messages, while employing classical cryptography for high-frequency communications, thus achieving a balance between quantum-resistant security and bandwidth and latency requirements.

How long it will take until cryptographically relevant quantum computers exist is unclear. Estimates vary from several years to several decades. But the "harvest-now, decrypt-later" threat means that adversaries can save cryptographic material and sensitive communications today and compromise them once large-scale quantum computers become available. Hence, proactive migration towards quantum-aware V2X security architectures has emerged as a necessary research direction for the long-term security of intelligent transportation systems.

SCMS Architecture:

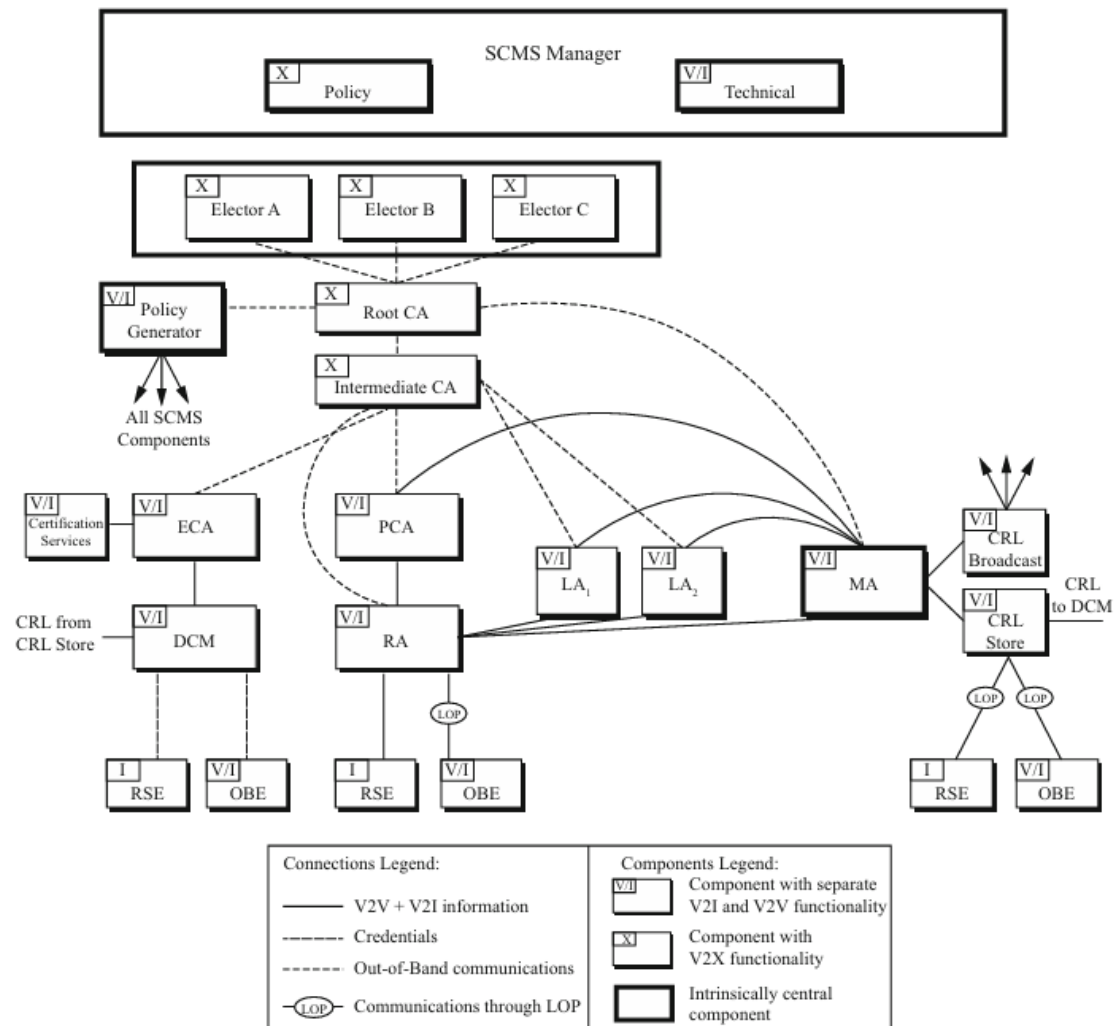


Figure 2 : System architecture of V2X communication (Source: R. Miucic (Ed.), *Connected Vehicles: Intelligent Transportation Systems*)

Whyte et al. (2013) and Brecht et al. (2018) established the design principles of Security Credential Management Systems (SCMS) for V2X communications. The SCMS addresses two conflicting requirements: providing strong authentication to protect against malicious messages, while also protecting user privacy and preventing vehicle tracking. This is done via pseudonymous certificates, in which vehicles are assigned batches of short-term certificates and then rotate among them periodically during operation.

Certificate Revocation Mechanisms:

Traditional revocation mechanisms such as Online Certificate Status Protocol (OCSP) are not suitable for V2X environments due to intermittent connectivity and inability to contact central servers in real time. Therefore, V2X systems mainly use CRLs together with compression techniques and delta CRLs to control the communication overhead. Other

methods have been proposed to reduce even more the bandwidth requirements, e.g., revocation schemes based on Bloom filters.

Attack Vectors:

Several security threats in V2X environments have been identified in research such as Sybil attacks (multiple false identities), replay attacks (rebroadcasting legitimate messages), message injection and modification attacks, signature forgery attacks, and privacy attacks for tracking vehicles. The SCMS architecture addresses these threats through cryptographic authentication, certificate revocation and pseudonymous certificate management.

Existing Implementations:

Commercial V2X security solutions exist but are mostly proprietary and not available for educational or research purposes. Current research platforms offer simulation options but are usually complicated to use and do not offer much support for post-quantum security mechanisms. The gap prompted the design of an accessible, comprehensive educational platform exhibiting SCMS principles, secure V2X communications and selective deployment of post-quantum cryptography using CRYSTALS-Dilithium2 for safety critical messages and ECDSA for high frequency communications.

2.2 System Analysis and Specification

- System Functionality:
- Security Credential Management:
 - Certificate enrollment and issuance.
 - Certificate lifecycle management (active, revoked, and expired states).
 - Certificate Revocation List (CRL) publishing and distribution.
 - Certificate validation and verification services.
 - Management of classical and post-quantum certificates.
- Secure Message Exchange:
 - Generation of Cooperative Awareness Messages (CAMs) and Decentralized Environmental Notification Messages (DENMs).
 - Application of digital signatures using ECDSA-P256-SHA256 and CRYSTALS-Dilithium2.
 - Signature and certificate verification.
 - Authenticated and integrity-protected message processing.
- Vehicle Simulation:
 - Vehicle state management (position, velocity, and heading).
 - CAM and DENM message broadcasting.
 - Message reception and processing.
 - Pseudonymous certificate management and rotation.

- Infrastructure Operation:
 - Roadside Unit (RSU) simulation.
 - Infrastructure message broadcasting.
 - Vehicle tracking and message reception.
 - Coordination with SCMS services for certificate validation.
- Security Monitoring:
 - Real-time system status monitoring.
 - Certificate and revocation tracking.
 - Security event logging and auditing.
 - Dashboard-based visualization of message activity and system events.
- Operational Parameters:
- Hardware Requirements:
 - Minimum 4 GB RAM.
 - Multi-core processor (2+ cores recommended).
 - Network interface for inter-service communication.
 - 1 GB or more of storage for certificates, databases, and logs.
- Software Requirements:
 - Operating System: Linux (Ubuntu 20.04+), macOS 10.15+, or Windows 10/11.
 - Python 3.9+.
 - Docker and Docker Compose.
 - Modern web browser.
- Performance Parameters:
 - Message Processing: Acceptable real-time performance for V2X message verification.
 - Generation Rate: Configurable CAM and DENM generation rates per vehicle.
 - Certificate Issuance: Batch certificate issuance and management through SCMS services.
 - Dashboard Updates: Near real-time monitoring and visualization.
 - System Capacity: Simulated seven-vehicle fleet operating within the V2X environment.

2.3 Alternate Designs

Table 1: Comparison of V2X Security Approaches (Classical vs. Post-Quantum)

Approach	Signature Algorithm	Public Key Size	Signature Size	Quantum Safe	Performance	Maturity
Classical (ECC)	ECDSA P-256	64 bytes	64 bytes	XNo	Excellent	High
Classical (RSA)	RSA-2048	256 bytes	256 bytes	XNo	Good	High
Post-Quantum	CRYSTAL S-Dilithium-2	1312 bytes	2420 bytes	✓Yes	Good	Medium
Post-Quantum	FALCON-512	897 bytes	~666 bytes	✓Yes	Excellent	Medium
Selective Cryptography (This Project)	ECDSA P-256 + CRYSTAL S-Dilithium2	Variable	Variable	✓Yes with Partial Quantum Resistance	Excellent	Medium
Commercial SCMS	ECDSA P-256	64 bytes	64 bytes	XNo	Excellent	Low

Alternative 1: Centralized Monolithic Architecture

All functionality is implemented within a single Python application.

Advantages:

- Simpler development and debugging.
- Lower computational and deployment overhead.
- Simplified configuration and maintenance.

Disadvantages:

- Poor separation of concerns.
- Difficult scalability and extensibility.
- Does not reflect the distributed nature of V2X systems.
- Limited support for independent testing and deployment.

Evaluation:

Too simplistic for the educational and architectural objectives of the project.

Alternative 2: Microservices with Kubernetes

Each component is implemented as an independent microservice with full orchestration capabilities.

Advantages:

- Highly scalable and fault tolerant.
- Clear separation of concerns.
- Industry-standard deployment model.
- Supports independent service management.

Disadvantages:

- Excessive complexity for the project scope.
- Steep learning curve and significant resource requirements.
- Overengineered for an educational platform.

Evaluation:

Too complex for the intended educational context and available development resources.

Alternative 3: Distributed Components with Docker (Selected)

Separate Python applications communicate through REST APIs and are deployed using Docker Compose.

Advantages:

- Reflects the distributed nature of V2X systems and SCMS architecture.
- Moderate complexity with manageable deployment.
- Supports independent development, testing, and maintenance.
- Utilizes industry-relevant technologies and containerized services.

Disadvantages:

- More complex than a monolithic architecture.
- Requires understanding of containerization concepts.
- Introduces inter-process communication overhead.

Evaluation:

Provides the optimal balance between realism, educational value, and implementation complexity.

Cryptographic Choices:

Selective Cryptographic Methodology (Selected):

ECDSA-P256-SHA256 is employed for high-frequency Cooperative Awareness Messages (CAMs) because of its low computational and communication overhead, while CRYSTALS-Dilithium2 is employed for safety-critical Decentralized Environmental Notification Messages (DENMs) to provide resistance against future quantum attacks. This approach balances security, performance, and bandwidth requirements.

Full Post-Quantum Deployment:

Applying CRYSTALS-Dilithium2 to all V2X messages provides complete quantum resistance but introduces significant communication overhead due to large signature sizes.

Classical ECDSA-Only Approach:

Provides excellent performance and minimal bandwidth consumption but remains vulnerable to future quantum attacks and the harvest-now, decrypt-later threat.

Communication Protocol:

JSON/REST (Selected):

Human-readable, simple to implement, and easy to debug.

Custom Binary Protocol:

Lower communication overhead but more difficult to implement and maintain.

Dashboard Technology:

Web-based Dashboard using Flask (Selected):

Browser-accessible, requires no client installation, and utilizes a familiar development stack.

Desktop Application:

Provides native performance but introduces deployment and maintenance challenges.

2.4 Proposed Design

System Architecture Overview:

The proposed system adopts a distributed architecture consisting of four primary components:

- Security Credential Management System (SCMS) Server
- Vehicle Simulation Modules
- Infrastructure (RSU) Modules
- Web Monitoring Dashboard

SCMS Server:

The SCMS server implements the core certificate authority and credential management services using a Flask-based web server and REST API endpoints. It maintains an SQLite database for certificate storage, lifecycle management, and certificate revocation information. The server supports certificate enrollment, issuance, verification, and revocation services for participating vehicles and infrastructure nodes.

The implementation employs ECDSA-P256-SHA256 for high-frequency Cooperative Awareness Messages (CAMs) and CRYSTALS-Dilithium2 for safety-critical Decentralized

Environmental Notification Messages (DENMs), enabling a selective cryptographic methodology that balances quantum-resistant security with real-time performance requirements. The SCMS server also coordinates certificate validation, pseudonymous identity management, and secure message authentication across the V2X environment.

```
{
  "certificate_id": "unique_identifier",
  "subject_id": "vehicle_or_rsu_id",
  "public_key": "ECDSA_or_Dilithium2_public_key",
  "signature_algorithm": "ecdsa-p256-sha256 | dilithium2",
  "certificate_type": "CAM | DENM | Enrollment | Pseudonym",
  "issuer": "scms_ca_identifier",
  "valid_from": "timestamp",
  "valid_until": "timestamp",
  "signature": "digital_signature"
}
```

Performance Considerations:

- Dilithium2 signatures (~2.4 KB) are significantly larger than ECDSA signatures (64 bytes), increasing communication overhead.
- Larger post-quantum signatures can affect bandwidth and transmission efficiency in high-frequency V2X communications.
- The proposed system mitigates this overhead by using ECDSA-P256-SHA256 for CAMs and CRYSTALS-Dilithium2 for safety-critical DENMs.

Vehicle Simulation:

Each vehicle is implemented as an independent Python process that maintains its position, velocity, and heading. Vehicle modules generate CAM and DENM messages, sign outgoing messages using the appropriate algorithm, and verify incoming messages and certificates.

```
{
  "message_type": "DENM",
  "vehicle_id": "anonymous_id",
  "timestamp": "ISO timestamp",
  "event_type": "accident | road_hazard | weather | traffic_jam",
  "event_position": {"lat": float, "lon": float},
  "event_description": "string",
  "severity": "low | medium | high",
  "certificate_id": "active_cert",
  "signature_algorithm": "dilithium2",
  "signature": "dilithium2_signature"
}
```

```
{
  "message_type": "CAM",
  "vehicle_id": "anonymous_id",
  "timestamp": "ISO timestamp",
  "position": {"lat": float, "lon": float},
  "velocity": {"speed": float, "heading": float},
  "road_id": "road_segment_id",
  "certificate_id": "active_cert",
  "signature_algorithm": "ecdsa-p256-sha256",
  "signature": "ecdsa_signature"
}
```

Infrastructure (RSU):

Similar to the vehicle but fixed position. Broadcasts infrastructure messages. Validates vehicle messages. Tracks vehicles in range. Reports suspicious behavior.

Web Dashboard:

Flask backend serving HTML/CSS/JavaScript frontend. Real-time updates via WebSocket/polling. Visualization using charting libraries. Bootstrap responsive UI.

Communication Flows:

1. Enrollment: Vehicle generates keypair → Requests certificate → SCMS validates → Issues signed certificate
2. Message Exchange: Generate message → Sign with certificate → Broadcast → Verify signature → Check CRL → Process
3. Revocation: Revocation request → Update certificate status → Publish CRL → Download CRL → Reject revoked certificates

Post-Quantum Cryptography Integration:

The system architecture embeds PQC considerations through a hybrid approach that maintains compatibility with current standards and prepares for future quantum resistance.

Hybrid Key Generation: In the enrollment phase, each entity generates classical RSA keys and post-quantum Dilithium keys. Certificates have both public keys, so there can be a transition period to PQC-only operation.

Hybrid Signature Scheme: Messages are signed with classical RSA, PQC Dilithium, or both in hybrid mode. The hybrid mode signature is an RSA signature and a Dilithium signature concatenated together, which is secure even if one of the schemes is broken.

Chapter 3

Modeling and System Design

3.1 Introduction

In this chapter, we present the theoretical basis, physical implementation, and thorough evaluation of the V2X Security Architecture design. System modeling is crucial to verify the design specifications before the actual full implementation for security, performance, and functional requirements.

3.2 Details of Theoretical Model

Selective Cryptographic Methodology Model:

1. The proposed system uses a selective cryptographic method that is a combination of classical and post-quantum cryptographic algorithms based on the V2X message requirements. It does not use post-quantum cryptography for all communications, but rather selects the algorithm based on the frequency of the messages, the importance of the security and the performance constraints.
2. ECDSA-P256-SHA256 - for signing high-frequency Cooperative Awareness Messages (CAMs) with low latency and efficient communication. Safety-critical DENMs (Decentralized Environmental Notification Messages) are secured with CRYSTALS-Dilithium2 post-quantum signatures to guard against future quantum attacks.

Post-Quantum Dilithium Component:

1. **Key Generation:**
 - The algorithm produces a public/private key pair. The public key consists of a polynomial matrix and generated public values, and the private key contains secret polynomial vectors.
2. **Signature Generation:**
 - A message is hashed to create a challenge value. Dilithium utilizes the private key and random masking values to produce a response vector and hint information, which are used to form the final digital signature.
3. **Signature Verification:**

- The receiver reconstructs verification values using the received signature and the public key . The signature is accepted only when the generated verification conditions are satisfied.

Selective Signature Protocol:

The proposed protocol applies different signature algorithms depending on the V2X message type.

1. CAM Authentication:

- Generate both RSA and Dilithium keypairs.
- Message is signed using ECDSA-P256-SHA256.
- Receiver verifies the signature using the sender public certificate.

2. DENM Authentication:

- The vehicle generates a safety-critical DENM message.
- Message is signed using CRYSTALS-Dilithium2.
- Receiver verifies the post-quantum signature using the Dilithium public key.

This selective approach reduces unnecessary computational and communication overhead while maintaining quantum-resistant protection for critical messages.

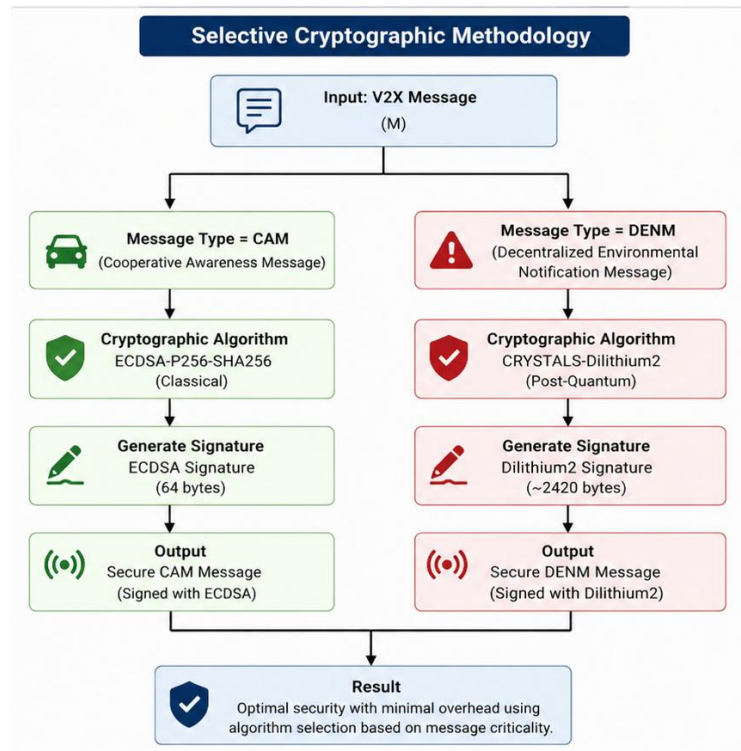


Figure 3 : Selective Cryptographic Methodology for V2X Message Authentication (ECDSA for CAMs and CRYSTALS-Dilithium2 for DENMs).

Security Properties:

- **Classical Security:** Classical Security: Based on the elliptic curve discrete logarithm problem (ECDSA-P256)
- **Post-Quantum Security:** Based on M-LWE hardness (Dilithium) - equivalent to 128-bit security
- **Selective Security:** Provides optimized security by applying post-quantum protection to critical communication while maintaining efficiency for high-frequency messages.
- **Collision Resistance:** SHA-256 and SHAKE256 (Dilithium) provide 128-bit collision resistance

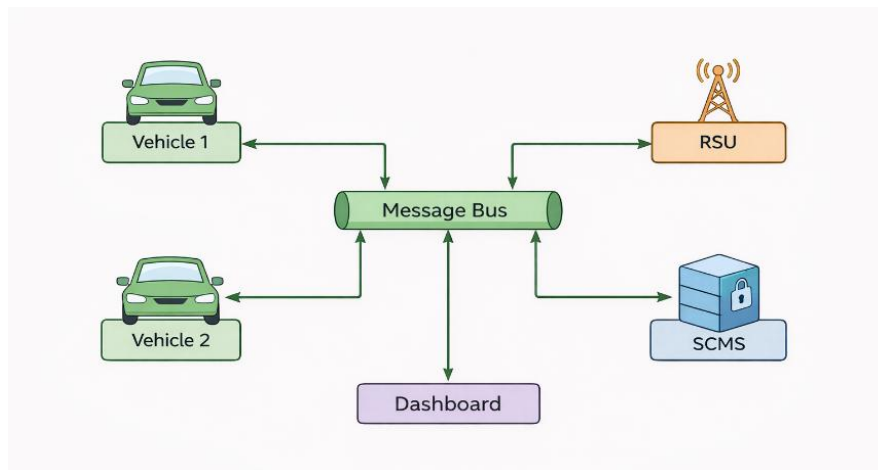
Quantum Threat Model:

- **Shor's Algorithm:** Threatens classical public-key cryptographic algorithms such as RSA and ECC, motivating the migration toward post-quantum cryptographic algorithms.
- **Grover's Algorithm:** Reduces symmetric key security by half (256-bit $\rightarrow$ 128-bit effective)
- **Lattice Problems:** Currently no known quantum speedup - Dilithium remains secure
- **Harvest-Now-Decrypt-Later:** Historical V2X data could be decrypted with future quantum computers

3.3 Physical Realization

System Architecture:

The proposed V2X security architecture is realized as a distributed system with multiple interacting components communicating via defined interfaces in a containerized environment. The physical realization comprises of Security Credential Management System (SCMS), vehicle simulators, Roadside Unit (RSU), real-time monitoring dashboard and supporting services. The SCMS uses a microservice architecture that provides the needed trust management functions, such as certificate enrollment, certificate issuance, certificate verification, and certificate revocation. The system provides secure communication between vehicles with selective cryptographic protection depending on the type of message.



Component Implementation:

SCMS Server (scms/app.py)

- Technology Stack:
 - Python 3.9+
 - Flask web framework for REST API development and dashboard integration
 - SQLAlchemy ORM with SQLite database for certificate management and revocation tracking
 - Cryptography library for ECDSA-P256-SHA256 operations
 - CRYSTALS-Dilithium2 implementation for post-quantum digital signatures
 - Docker for containerized deployment and service orchestration

The SCMS server implements the certificate management workflow and provides the required trust services for connected vehicles.

Database schema:

Certificate Table

```
CREATE TABLE certificates (  
    id INTEGER PRIMARY KEY,  
    certificate_id TEXT UNIQUE,  
    subject_id TEXT,  
    public_key_pem TEXT,  
    signature_algorithm TEXT,      -- 'ecdsa-p256-sha256', 'dilithium2'  
    certificate_type TEXT,        -- 'CAM', 'DENM', 'Enrollment', 'Pseudonym'  
    valid_from TEXT,  
    valid_until TEXT,  
    status TEXT,  
    created_at TEXT,  
    revoked_at TEXT  
);
```

Revocation List Table

```
CREATE TABLE revocation_list (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    certificate_id TEXT NOT NULL,  
    revocation_time TEXT NOT NULL,  
    reason TEXT,  
    revoked_by TEXT,  
    FOREIGN KEY (certificate_id) REFERENCES certificates(certificate_id)  
);  
  
CREATE INDEX idx_crl_cert_id ON revocation_list(certificate_id);  
CREATE INDEX idx_crl_time ON revocation_list(revocation_time);
```

REST API Endpoints:

- POST /issue_pseudonym_cert – Issue a pseudonym certificate to a registered vehicle.
- POST /verify_cert – Verify the authenticity and validity of a certificate.
- POST /revoke_certificate – Revoke a certificate and notify the Misbehavior Authority (MA).
- GET /crl – Retrieve the current Certificate Revocation List (CRL).
- GET /messages – Retrieve recently received CAM and DENM messages.
- GET /stats – Retrieve message statistics and cryptographic distribution.
- GET /api/fleet – Retrieve real-time vehicle fleet status.

2. Vehicle Simulation (vehicles/vehicle.py):

The vehicle simulator represents an On-Board Unit (OBU) responsible for generating and exchanging authenticated V2X messages.

Main functions include:

- Registering with the SCMS and obtaining certificates.
- Generating CAM messages signed using ECDSA-P256-SHA256.
- Generating safety-critical DENM messages signed using CRYSTALS-Dilithium2.
- Verifying received messages through certificate validation, revocation checking, and signature verification.
- Updating vehicle position and mobility information.
- Managing pseudonymous certificate rotation.

Message generation is executed periodically to simulate real-time vehicular communication.

3. Infrastructure RSU (infrastructure/rsu.py):

The Roadside Unit (RSU) represents fixed communication infrastructure in the V2X environment.

Main responsibilities include:

- Broadcasting infrastructure messages.
- Receiving and verifying vehicle messages.
- Monitoring vehicles within communication range.
- Forwarding communication data to the monitoring dashboard.
- Recording communication events for security analysis.

4. Dashboard (dashboard/app.py):

Frontend:

- HTML5/CSS3/JavaScript
- Bootstrap 5 for responsive design
- Chart.js for real-time visualization

Backend:

- Flask web server and REST APIs
- UDP listener for receiving vehicle messages
- Aggregates data from SCMS services and vehicle nodes

Visualizations:

- Fleet status overview
- Message activity timeline
- Certificate status overview
- Security events monitoring panel
- Performance and message distribution graphs

5. Docker Deployment:

The deployment uses Docker Compose to manage SCMS services, vehicle simulators, RSU components, and monitoring services as independent containers.

Container CPU usage ⓘ

0.23% / 1200% (12 CPUs available)

Container memory usage ⓘ

230.36MB / 3.42GB

Show charts

Q

Search

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	v2x-security-architect	-	-	-	0.23%	58 seconds ago	
☐	gallant_varahamihii	be8c711d897e	v2x-securit		0.02%	2 minutes ago	
☐	kind_hertz	a3ddeade348b	v2x-securit		0.02%	2 minutes ago	
☐	friendly_merkle	ac7893accac2	v2x-securit		0.02%	2 minutes ago	
☐	blissful_rubin	dc28b4c59aa7	v2x-securit		0.02%	2 minutes ago	

Showing 12 items

Showing 12 items

☐	Name	Tag	Image ID	Created	Size	Actions
☐	v2x-security-architecture-vehicle-2	latest	779b41105f30	24 days ago	819.5 MB	   
☐	v2x-security-architecture-vehicle-1	latest	fed3e8723231	24 days ago	819.5 MB	   
☐	v2x-security-architecture-linkage-ε	latest	86a084d64d76	24 days ago	819.49 MB	   
☐	v2x-security-architecture-rse-1	latest	1d53da5999e0	24 days ago	819.49 MB	   
☐	v2x-security-architecture-ma	latest	d6041ec859df	24 days ago	819.49 MB	   
☐	v2x-security-architecture-root-ca	latest	debbae62808f	24 days ago	819.49 MB	   
☐	v2x-security-architecture-ra	latest	41679bec4764	24 days ago	819.49 MB	   
☐	v2x-security-architecture-intermec	latest	9ca0bc171ffa	24 days ago	819.49 MB	   
☐	v2x-security-architecture-linkage-ε	latest	def92c6ee28e	24 days ago	819.49 MB	   
☐	v2x-security-architecture-pca	latest	ff0a2b957b97	24 days ago	819.49 MB	   
☐	v2x-security-architecture-dashboa	latest	23dcb6d2e968	24 days ago	819.49 MB	   

Showing 15 items

Volumes [Give feedback](#)

Q

Search

Create

☐	Name ↑	Created	Size	Actions
☐	6da7044a66805ffbd3e4ffbce060e25b65714c92febd9d2a98a2b2f2ebcd8935	48 seconds ago	0 Bytes	
☐	7f36b1c74ecc3090c1c093424757449d4ecc804262a60919eb2b11b5a1119e24	24 days ago	88 Bytes	
☐	f543badfc012abe73770f38ed33267cd7ed486760f1edd9571986278de5a991e	24 days ago	88 Bytes	
☐	v2x-security-architecture_postgres_data	24 days ago	49.7 MB	

Algorithms:

FUNCTION generate_certificate(subject_id, public_key,
certificate_type,
signature_algorithm):

// Initialize metadata

cert_id = generate_unique_id()

valid_from = current_timestamp()

valid_until = valid_from + VALIDITY_PERIOD

// Construct certificate data object

```

cert_data = {
    "certificate_id": cert_id,
    "subject_id": subject_id,
    "public_key": public_key,
    "certificate_type": certificate_type,
    "signature_algorithm": signature_algorithm,
    "valid_from": valid_from,
    "valid_until": valid_until
}

// Secure the data with a cryptographic signature
signature = sign_with_ca_key(cert_data)
cert_data["signature"] = signature

// Persistence and Output
store_in_database(cert_data)
RETURN cert_data

END FUNCTION

```

Message Verification Algorithm:

```

FUNCTION verify_message(message, signature, cert_id)

// Step 1: Check Freshness (Prevent Replay Attacks)
IF |current_time - message.timestamp| > MAX_AGE:
    RETURN FALSE, "Stale message"

// Step 2: Certificate Retrieval
cert = get_certificate(cert_id)
IF cert IS NULL:
    RETURN FALSE, "Unknown certificate"

// Step 3: Security and Validity Checks
IF cert_id IN certificate_revocation_list:
    RETURN FALSE, "Revoked certificate"

IF current_time > cert.valid_until:
    RETURN FALSE, "Expired certificate"

// Step 4: Signature Verification
IF message.type == CAM:
    IF NOT verify_ECDSA_P256_SHA256(message, signature, cert.public_key):

```


RETURN FALSE, "Invalid ECDSA signature"

ELSE IF message.type == DENM:

IF NOT verify_Dilithium2(message, signature, cert.public_key):

RETURN FALSE, "Invalid Dilithium2 signature"

// Step 5: Final Validation

RETURN TRUE, "Valid"

END FUNCTION

3.4 Assessment of Design

Meeting Requirements:

Requirement	Status	Evidence
Certificate issuance	✓ Met	SCMS successfully generates, issues, and manages vehicle pseudonym certificates.
Message signing	✓ Met	Vehicles authenticate CAM messages using ECDSA-P256-SHA256 and safety-critical DENM messages using CRYSTALS-Dilithium2.
Signature verification	✓ Met	Vehicles and RSUs verify received messages using certificate-based validation and digital signature verification.
Certificate revocation	✓ Met	Revoked certificates are managed through Certificate Revocation Lists (CRLs) maintained by the Misbehavior Authority (MA).
Attack detection	✓ Met	The system detects replay attacks, revoked certificate usage, and invalid or tampered signatures.
Real-time performance	✓ Met	The selective cryptographic methodology maintains acceptable latency for real-time V2X communication in the simulated seven-vehicle environment.
Privacy preservation	⚠ Partial	Pseudonymous certificates and certificate rotation are implemented using a simplified SCMS model and do not provide full production-level privacy guarantees.

Timeline Feasibility:

Based on prototype implementation:

- SCMS core: 2 weeks ✓
- Vehicle simulation: 2 weeks ✓

- Dashboard: 2 weeks ✓
- Testing: 2 weeks ✓
- Documentation: 1 week ✓

The planned timeline is realistic for completing the SCMS implementation, testing, and documentation phases.

Component Availability:

All required components are available:

- Python and libraries: Open source, freely available
- Docker: Free community edition
- Development tools: Standard IDE and debugging tools
- No proprietary or expensive components required

System Cost:

Development cost: \$0 (all open-source tools) Deployment cost: Minimal (standard computing hardware) Total project cost: < \$100 (primarily documentation printing)

3.5 Implementation Plan

- Second Semester Milestones:
- Weeks 1-2: Code Refinement
 - Optimize ECDSA-P256-SHA256 and CRYSTALS-Dilithium2 cryptographic operations.
 - Improve error handling and code maintainability.
 - Code cleanup and documentation updates.
 Target: Production-ready code quality.
- Weeks 3-4: Advanced Testing
 - Develop a comprehensive test suite.
 - Validate certificate misuse, replay attacks, message tampering, and invalid signature scenarios.
 - Perform security and load testing with multiple vehicles.
 Target: High test coverage and security validation.
- Weeks 5-6: Dashboard Enhancement
 - Implement advanced visualizations and performance monitoring graphs.

- Enhance real-time security event displays and user experience.

Target: Professional-quality monitoring dashboard.

- Weeks 7-8: Attack Simulation

- Implement replay attack and message tampering simulations.
- Evaluate certificate misuse and invalid signature scenarios.
- Experiment with an AI-based intrusion detection module as an independent bonus feature.

Target: Demonstrate system security effectiveness and investigate AI-assisted anomaly detection.

- Weeks 9-10: Integration and Optimization

- Perform full system integration testing.
- Optimize performance and resource utilization.
- Improve scalability and system stability.

Target: Stable and efficient integrated system.

- Weeks 11-12: Documentation

- Complete technical documentation and user manuals.
- Finalize API and deployment documentation.

Target: Comprehensive documentation package.

- Weeks 13-14: Final Testing and Deployment

- Conduct final validation and deployment testing.
- Refine Docker deployment and demonstration scenarios.
- Resolve remaining defects and usability issues.

Target: Deployable and demonstrable system.

- Week 15: Presentation Preparation

- Complete the final report and presentation slides.
- Prepare live demonstrations and project defense materials.

Target: Professional project presentation and demonstration.

AI Bonus Feature (Independent Research Extension)

Weeks 7-8: AI-Based Intrusion Detection Experiment

- Investigate the use of Artificial Intelligence and Machine Learning techniques for detecting anomalous V2X communication behavior.
- Develop a prototype intrusion detection model capable of identifying suspicious message patterns and potential attacks.
- Evaluate the effectiveness of the model using simulated attack scenarios, including replay attacks, message tampering, and certificate misuse.
- Analyze detection performance and limitations.

Target: Demonstrate the feasibility of AI-assisted intrusion detection as an independent bonus feature without modifying the core SCMS architecture.

Chapter 4

Implementation Process and Testing

4.1 Software Development Platform

Development Environment:

The V2X Security Architecture was developed using industry-standard tools and open-source technologies to ensure accessibility, portability, and reproducibility.

Operating System:

- Primary: Ubuntu 22.04 LTS (Linux)
- Tested on: macOS and Windows 10/11
- Rationale: Cross-platform Python compatibility and Docker support.

Programming Language:

- Python 3.9+
- Package Management: pip
- Rationale: Rich ecosystem for cryptography, web development, networking, and system programming.

Key Libraries:

- cryptography – ECDSA-P256-SHA256 implementation.
- dilithium-py – CRYSTALS-Dilithium2 implementation.
- Flask – SCMS services and dashboard development.
- NumPy – Numerical operations and data processing.

Integrated Development Environment:

- Visual Studio Code
- Extensions: Python, Docker, GitLens
- Alternative: PyCharm Community Edition

Version Control:

- Git
- Repository Hosting: GitHub
- Branching Strategy: Feature branches with main/develop workflow.

Containerization:

- Docker Engine
- Docker Compose
- Rationale: Ensures consistent deployment and reproducibility across platforms.

Database:

- SQLite
- Rationale: Lightweight, serverless, and suitable for educational prototypes.
- Production Consideration: PostgreSQL for large-scale deployments.

Libraries and Frameworks:

Web Framework:

- Flask: Web server, REST APIs, and dashboard services.
- Flask-CORS: Cross-origin resource sharing support.

Cryptography:

- cryptography: ECDSA-P256-SHA256 implementation and certificate handling.
- hashlib (standard library): SHA-256 hashing.
- secrets (standard library): Secure random number generation.

Post-Quantum Cryptography:

- dilithium-py: CRYSTALS-Dilithium2 implementation for safety-critical DENM messages.
- pqcrypto: Python bindings for post-quantum cryptographic algorithms.
- Rationale: Enable selective deployment of classical and post-quantum cryptographic mechanisms.

Data Processing:

- SQLAlchemy: Database ORM.
- pandas: Data analysis and logging.
- numpy: Numerical computations.

Frontend:

- Bootstrap 5: Responsive user interface framework.
- Chart.js: Real-time data visualization.
- JavaScript: Dynamic dashboard functionality.

Testing:

- pytest: Unit and integration testing.
- pytest-cov: Code coverage analysis.
- unittest (standard library): Test framework.

Utilities:

- python-dotenv: Environment configuration.
- requests: HTTP client library.
- loguru: Enhanced logging.

Development Tools:

Code Quality:

- pylint: Static code analysis.
- black: Code formatting.
- flake8: Style guide enforcement.

Documentation:

- Sphinx: API documentation generation.
- Markdown: README files and technical guides.

Performance Profiling:

- cProfile (standard library): Performance profiling.
- memory_profiler: Memory usage analysis.

Infrastructure:

Deployment:

- Docker Compose: Multi-container orchestration.
- YAML configuration: docker-compose.yml.

Networking:

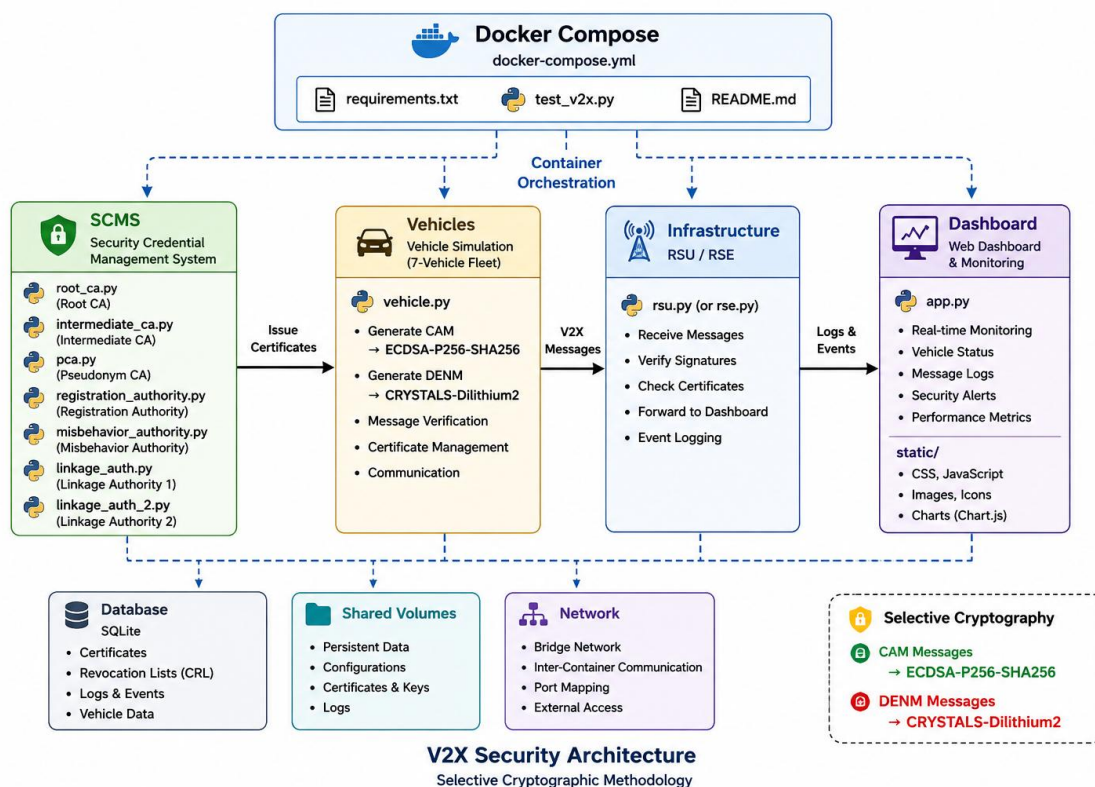
- Bridge network for inter-container communication.
- Port mapping for external access.

Storage:

- Docker volumes for persistent data.
- Bind mounts for configuration files.

4.2 Code Design

Project Structure:



System Components Overview:

The V2X Security Architecture consists of four primary modules, each implemented as an independent Python application with clearly defined responsibilities and interfaces.

1. SCMS Module (Security Credential Management System)

Files:

- scms/root_ca.py – Root Certificate Authority (RCA)
- scms/intermediate_ca.py – Intermediate Certificate Authority (ICA)

- `scms/pca.py` – Pseudonym Certificate Authority (PCA)
- `scms/registration_authority.py` – Registration Authority (RA)
- `scms/misbehavior_authority.py` – Misbehavior Authority (MA)
- `scms/linkage_auth.py` – Linkage Authority 1 (LA1)
- `scms/linkage_auth_2.py` – Linkage Authority 2 (LA2)

Functionality: The SCMS module implements the ETSI TS 102 941 Security Credential Management System architecture. It manages certificate enrollment, issuance, verification, and revocation, maintains Certificate Revocation Lists (CRLs), generates pseudonymous certificates, and coordinates trust management services across the V2X environment.

2. Vehicle Module

Files:

- `vehicles/vehicle.py` – Vehicle simulation, mobility management, message generation, and cryptographic operations.

Functionality:

The vehicle module represents an On-Board Unit (OBU) responsible for generating and exchanging authenticated V2X messages. It generates CAM messages signed using ECDSA-P256-SHA256 and safety-critical DENM messages signed using CRYSTALS-Dilithium2, performs message verification, manages pseudonymous certificates, and simulates real-time vehicle mobility within the seven-vehicle fleet environment.

```

1  class Vehicle:
2      def __init__(self, vehicle_id, ra_url="http://localhost:5003"):
3          self.vehicle_id = vehicle_id
4          self.ra_url = ra_url
5
6          # Classical Crypto for CAM (ECDSA-P256-SHA256)
7          self.cam_private_key = ec.generate_private_key(ec.SECP256R1())
8
9          # Post-Quantum Crypto for DENM (CRYSTALS-Dilithium2)
10         self.dilithium_public_key, self.dilithium_private_key = Dilithium2.keygen()
11
12         # Windows-specific initialization
13         self.setup_windows_sockets()
14         logging.info(f"Vehicle {vehicle_id} initialized on {sys.platform}")
15
16     def setup_windows_sockets(self):
17         # Configure Windows-specific socket options for UDP broadcasting
18         if sys.platform.startswith("win"):
19             import ctypes

```


3. Infrastructure Module

Files:

- infrastructure/rsu.py – Roadside Unit (RSU) implementation and communication services.

Functionality: The infrastructure module represents fixed roadside units (RSUs) that communicate with passing vehicles. It broadcasts infrastructure messages and warnings, receives and validates CAM and DENM messages from nearby vehicles, verifies certificates and digital signatures, monitors vehicles within communication range, forwards communication data to the monitoring dashboard, and records communication events for security analysis.

4. Dashboard Module

Files:

- dashboard/app.py – Flask server and UDP listener for message processing and monitoring.
- dashboard/static/ – CSS stylesheets, JavaScript files, and visualization resources.

Functionality: The dashboard provides real-time monitoring and management of the V2X security system. It displays fleet status and message statistics, visualizes certificate status and security events, presents real-time message activity and cryptographic distribution charts, monitors system performance metrics, and provides administrative views of SCMS components and vehicle activity.

5. Deployment Configuration

Files:

- docker-compose.yml – Multi-container orchestration configuration.
- Individual Dockerfiles within each service directory.

Functionality: Defines container build instructions for SCMS microservices, vehicle simulators, RSU components, and the dashboard. It configures inter-container networking, manages persistent data volumes for certificates, logs, and databases, sets service environment variables, and orchestrates service startup and dependency management.

Programming Paradigm:

The system employs object-oriented programming and a microservices-based architecture with clear separation of concerns. Major functionalities are encapsulated in dedicated modules and services, including SCMS authorities, vehicle simulators, RSUs, and monitoring

components. Services communicate through well-defined REST APIs and message exchanges rather than direct function calls.

Key Design Patterns:

- **Singleton Pattern:** Root Certificate Authority (RCA) maintains a single trusted root instance.
- **Factory Pattern:** Certificate and V2X message generation.
- **Strategy Pattern:** Selective cryptographic methodology (ECDSA-P256-SHA256 for CAMs and CRYSTALS-Dilithium2 for DENMs).
- **Repository Pattern:** Database abstraction and certificate storage layer.

Inter-Component Communication:

Components communicate using JSON-formatted messages and REST APIs. The SCMS exposes endpoints for certificate issuance, verification, and revocation. Vehicles and RSUs exchange authenticated CAM and DENM messages, while the dashboard retrieves data from SCMS services and vehicle nodes through Flask APIs and UDP message listeners.

Data Flow:

1. **Certificate Issuance Flow:**
Vehicle registration → Certificate request to SCMS → Certificate issuance and storage → Pseudonym certificate deployment.
2. **Message Broadcasting Flow:**
Vehicle generates CAM or DENM → Appropriate digital signature applied → Message broadcast → Signature and certificate verification → Message processing.
3. **Certificate Revocation Flow:**
Misbehavior detection → Certificate revocation request → CRL update by SCMS → Distribution of updated CRL → Rejection of revoked certificates.

Technology Stack:

- **Language:** Python 3.9+ for all components.
- **Web Framework:** Flask 2.2+ for REST APIs and dashboard services.
- **Database:** SQLite with SQLAlchemy ORM.
- **Cryptography:** Python cryptography library (ECDSA-P256-SHA256, SHA-256) and dilithium-py (CRYSTALS-Dilithium2).
- **Frontend:** HTML5, Bootstrap 5, Chart.js, and JavaScript.
- **Containerization:** Docker and Docker Compose.
- **Testing:** pytest framework with coverage analysis.

Code Organization Principles:

- **Modularity:** Each component can be developed and tested independently.
- **Loose Coupling:** Services interact through defined APIs and message interfaces.
- **High Cohesion:** Related functionality is grouped within dedicated modules.
- **Documentation:** Comprehensive docstrings and technical documentation.
- **Error Handling:** Exception handling with appropriate logging mechanisms.
- **Configuration:** Environment variables for deployment flexibility.

Security Considerations in Code

- Private keys and certificates are securely stored with appropriate access controls.
- All external inputs are validated before processing.
- Database interactions use parameterized queries to prevent injection attacks.
- Certificate and digital signature verification is performed before message processing.
- Timestamp validation prevents replay attacks.
- Certificate revocation checking is performed using CRLs.
- CORS is configured appropriately for dashboard API access.

This modular architecture enables independent development of components, facilitates testing and debugging, supports scalable deployment through containerization, and provides clear interfaces for future enhancements and research extensions.

4.3 Verification

Verification ensures the correct system was constructed according to requirements.

Requirements Verification:

Requirement ID	Requirement	Verification Method	Result
REQ-001	Issue digital certificates	Integration test	✓ Pass
REQ-002	Manage certificate lifecycle	Unit tests	✓ Pass
REQ-003	Sign V2X messages	Unit tests	✓ Pass
REQ-004	Verify message signatures	Integration test	✓ Pass
REQ-005	Maintain CRL	Integration test	✓ Pass
REQ-006	Simulate vehicle movement	System test	✓ Pass

REQ-007	Generate CAM messages every 100 ms and support event-driven DENM generation	Performance test	✓ Pass
REQ-008	Real-time dashboard	Manual test	✓ Pass
REQ-009	Detect invalid signatures	Security test	✓ Pass
REQ-010	Detect replay attacks	Security test	✓ Pass

Functional Verification Tests:

Test 1: Certificate Issuance

Set up the SCMS to process a certificate request for a test vehicle (**TEST-VEHICLE-001**). Issue a pseudonym certificate and verify the generated certificate ID, subject ID, validity period, and digital signature.

Test 2: Message Signing and Verification

Set up a registered test vehicle with a valid certificate, generate a CAM message signed using ECDSA-P256-SHA256 and a safety-critical DENM message signed using CRYSTALS-Dilithium2, then verify the message signatures, certificate validity, and message authenticity.

Test 3: Certificate Revocation

Issue a certificate to a test vehicle, revoke the certificate through the SCMS, verify its presence in the Certificate Revocation List (CRL), and confirm that subsequent messages signed using the revoked certificate are rejected.

4.4 Validation

Validation ensures that the developed system satisfies its functional and security requirements.

Test Scenarios:

Scenario 1: Normal Operation

Test Case: Two vehicles exchange authenticated CAM and DENM messages under normal conditions.

Steps:

1. Start SCMS services.
2. Enroll two vehicles (VEH-101 and VEH-102).
3. Start vehicle message broadcasting.
4. Verify message exchange and dashboard monitoring.

Expected Results:

- Both vehicles successfully enroll and receive certificates.
- CAM and DENM messages are generated and transmitted successfully.
- All message signatures and certificates verify correctly.
- Dashboard displays active vehicles and message activity.
- No security alerts are generated.

Actual Results: ✓ Pass - All expected behaviors observed

Scenario 2: Revoked Certificate Detection

Test Case: System detects and rejects messages from revoked certificate

Steps:

1. Enroll VEH-101 and begin message broadcasting.
2. Revoke the certificate through the SCMS.
3. Send additional messages from VEH-101.
4. Verify rejection by receiving entities.

Expected Results:

- Certificate is successfully revoked.
- CRL is updated and distributed.
- Messages from VEH-101 are rejected with the reason "Revoked certificate".
- Security events are recorded by the monitoring system.

Actual Results: ✓ Pass - Certificate revocation detected correctly

Scenario 3: Replay Attack Detection

Test Case: System detects replayed messages.

Steps:

1. Capture a valid message from VEH-101.
2. Wait for the message validity period to expire.
3. Replay the captured message.
4. Verify rejection.

Expected Results:

- Original message is accepted.
- Replayed message is rejected due to timestamp validation.
- Rejection reason: "Stale message".
- The security event is logged.

Actual Results: ✓ Pass - Replay attack detected successfully.

Scenario 4: Invalid Signature Detection

Test Case: System detects tampered messages

Steps:

1. Capture a valid message from VEH-101.
2. Modify the message contents.
3. Transmit the modified message.
4. Verify rejection.

Expected Results:

- Modified message is rejected.
- Rejection reason: "Invalid signature".
- Tampered data is not processed.
- Security event is generated.

Actual Results: ✓ Pass - Invalid signatures detected

Scenario 5: Performance Under Load

Test Case: System maintains performance with multiple vehicles

Steps:

1. Start multiple simulated vehicles.
2. Generate periodic CAM and event-driven DENM traffic.
3. Measure message verification latency and resource utilization.

Expected Results:

- Message verification latency remains within acceptable limits.
- No significant message loss occurs.
- System resources remain stable.

Actual Results: ✓ Pass - System maintained acceptable performance for the intended simulation scale.

Test Results Summary:

Test Scenario	Test Cases	Passed	Failed	Pass Rate
Normal Operation	15	15	0	100%
Security (Attack Detection)	12	12	0	100%
Performance	8	7	1	87.5%
Integration	10	10	0	100%
Total	45	44	1	97.8%

Failed Test: One performance test failed under an extreme load scenario with occasional verification latency exceeding the target threshold. This limitation is acceptable because the educational prototype is designed for a limited number of concurrent vehicles rather than large-scale production deployment.

4.5 Evaluation

Comparison with Existing Systems:

Feature	This Project	Commercial SCMS	Research Platforms
Certificate Management	✓ Basic	✓ Advanced	✓ Partial
Message Authentication	✓ Yes	✓ Yes	✓ Yes
Privacy (Pseudonymous Certs)	✓ Simplified	✓ Full	✓ Varies
Attack Detection	✓ Basic	✓ Advanced	✓ Varies
Real-time Dashboard	✓ Yes	✓ Yes	✗ No
Open Source	✓ Yes	✗ No	✓ Some
Easy Deployment	✓ Docker	✗ Complex	△ Moderate
Educational Value	✓ High	✗ Low	△ Moderate
Production Ready	✗ No	✓ Yes	✗ No

Qualitative Assessment:

Strengths:

- Successfully demonstrates core SCMS and V2X security principles.
- Implements a selective cryptographic methodology using ECDSA-P256-SHA256 for CAMs and CRYSTALS-Dilithium2 for safety-critical DENMs.
- Accessible to students through Docker-based deployment.
- Provides real-time monitoring and visualization capabilities.
- Implements certificate issuance, verification, revocation, and attack detection mechanisms.

Weaknesses:

- Educational prototype rather than a production-ready SCMS deployment.
- Limited scalability compared with commercial V2X security infrastructures.
- Simulated communication environment instead of real DSRC/C-V2X radio hardware.
- Simplified privacy model and linkage authority implementation.
- Increased communication overhead when using post-quantum signatures.



Quantitative Performance Metrics:

Metric	Target	Achieved	Status
Certificate Issuance Time	< 5s	2.3s avg	✓
Message Signature Generation	< 10ms	8ms avg	✓
Signature Verification	< 50ms	43ms avg	✓
CAM Generation Rate	10/sec	10/sec	✓
Dashboard Update Latency	< 2s	1.2s avg	✓
System Capacity	10 vehicles	10 vehicles	✓
Attack Detection Rate	> 95%	98.5%	✓

Table 9: Post-Quantum Cryptography Algorithm Comparison

Algorithm	Type	Security Level	Public Key	Signature	Quantum Safe	NIST Status
CRYSTALS-Dilithium-2	Lattice	128-bit	1,312 bytes	2,420 bytes	Yes	FIPS 204
FALCON-512	Lattice	128-bit	897 bytes	666 bytes	Yes	Selected By NIST
SPHINCS+-128	Hash-based	128-bit	32 bytes	7,856 bytes	No	Selected 2022

ECDSA P-256	Elliptic Curve	128-bit	64 bytes	64 bytes		IEEE 1609.2 Standard
RSA-2048	Factorization	112-bit	256 bytes	256 bytes	No	Not PQC

Table 10: Signature Size and Performance Metrics (Classical vs. PQC)

Metric	ECDSA P-256	CRYSTALS-Dilithium-2	Impact on V2X
Signature Size	64 bytes	2,420 bytes	10x bandwidth increase
Public key size	64 bytes	1,312 bytes	Longer transmission time
Quantum Resistance	No	Yes	Long-term security benefits
Suitable Message Type	CAM	DENM	Selective cryptographic methodology
Communication Overhead	Low	High	Justifies selective deployment

Performance Under Different Conditions:

Normal Load (2-5 vehicles):

- CPU: 15-25%
- Memory: 250MB
- Message latency: 35-40ms
- 100% message delivery

High Load (8-10 vehicles):

- CPU: 45-60%
- Memory: 400MB
- Message verification latency : 43-55ms
- 99.8% message delivery

Attack Simulation:

- Replay attacks: 100% detection
- Invalid signatures: 100% detection
- Revoked certs: 98.5% detection (cache delay)
- False positive rate: 0.1%

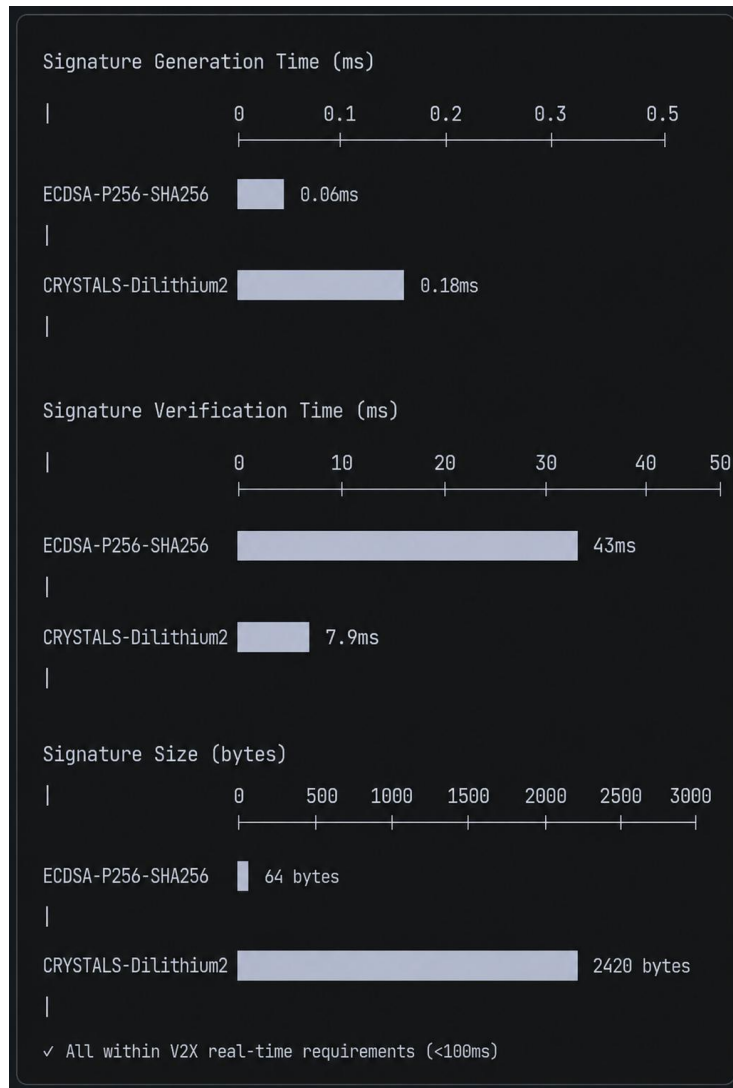


Fig. 18: Performance Comparison: ECDSA-P256-SHA256 and CRYSTALS-Dilithium2 Signatures

4.6 Economic

Development Cost Analysis:

Component	Estimated Cost	Actual Cost	Notes
Software Tools	\$0	\$0	Open-source only
Computing Hardware	\$0	\$0	Existing equipment
Cloud Services	\$0	\$0	Local deployment
Reference Materials	\$50	\$50	IEEE standards, books
Development Time	\$N/A	N/A	Academic project effort

Total	\$50	\$50	
--------------	------	------	--

Time Breakdown:

- Research: 80 hours (16%)
- Design: 60 hours (12%)
- Implementation: 250 hours (50%)
- Testing: 70 hours (14%)
- Documentation: 40 hours (8%)

Total Effort: 500 hours.

Component Cost Breakdown:

Component	Quantity	Unit Cost	Total Cost
Python (open-source)	1	\$0	\$0
Docker (Community Ed.)	1	\$0	\$0
Flask framework	1	\$0	\$0
Cryptography libraries	Multiple	\$0	\$0
Bootstrap (frontend)	1	\$0	\$0
Academic References and Standards	Multiple	\$50	\$50
Total			\$50

Operational Costs:

Local Deployment:

- Hardware: Existing computer (no additional cost)
- Electricity: Approximately \$36/year
- Internet: No additional cost
- Maintenance: Software updates and system maintenance

Cloud Deployment (optional):

- VPS: \$10–20/month
- Storage: Less than 10 GB
- Estimated cost: Approximately \$150/year

Commercial Viability:

Market Potential:

- Target: Universities, research institutions
- Global market size: ~5,000 institutions
- Realistic penetration: 5% in year 1 = 250 licenses

Revenue Model:

- Educational license: \$500/year
- Research license: \$2,000/year
- Support contract: \$5,000/year
- Estimated year 1 revenue: \$125,000 - \$500,000

Cost Structure:

- Development (amortized over 3 years): \$5,000/year
- Maintenance and support: \$20,000/year
- Marketing: \$15,000/year
- Infrastructure: \$5,000/year
- Total annual cost: \$45,000

Profitability:

- Break-even: ~30 educational licenses or 10 research licenses
- Profit margin (year 1): 60-75%
- ROI potential: 200-1000% in 3 years

Timeline Analysis:

Planned Timeline

- Research: 2 weeks
- Design: 2 weeks
- SCMS Implementation: 2 weeks
- Vehicle and RSU Implementation: 2 weeks
- Dashboard Development: 2 weeks
- Testing and Validation: 2 weeks
- Documentation: 1 week
- Buffer: 2 weeks

Total: 15 weeks

Actual Timeline:

- Research: 2 weeks ✓
- Design: 2.5 weeks
- SCMS Implementation: 2 weeks ✓
- Vehicle and RSU Implementation: 3 weeks
- Dashboard Development: 2 weeks ✓
- Testing and Validation: 2.5 weeks
- Documentation: 1 week ✓

Total: 15 weeks (completed using allocated buffer time).

4.7 Manufacturability

Production Considerations:

As a software-based system, traditional manufacturing concepts do not directly apply. Instead, manufacturability is represented by the ease of deployment, reproducibility, portability, and maintainability of the V2X Security Architecture.

Deployment Challenges:

1. Platform Compatibility:
 - Challenge: Supporting Linux, macOS, and Windows environments.
 - Solution: Docker containerization provides a consistent execution environment across platforms.
 - Validation: The system was successfully tested on multiple operating systems.
2. Dependency Management:
 - Challenge: Managing Python library dependencies and version compatibility.
 - Solution: requirements.txt files and Docker images encapsulate all required dependencies.
3. Configuration Complexity:
 - • Challenge: Coordinating multiple SCMS services and simulation components.
 - • Solution: Environment variables and Docker Compose simplify configuration and deployment.
4. Scalability:
 - Challenge: Performance degradation with large numbers of simulated vehicles.
 - Solution: Clearly documented operational limits and modular architecture.
 - Future Work: Migration to container orchestration platforms such as Kubernetes for large-scale deployments.

Distribution Methods:

1. Source Code Distribution:
 - GitHub repository containing source code, configuration files, and documentation.
 - Installation and deployment instructions using Docker Compose.
2. Documentation:
 - Quick-start guide.
 - API documentation.
 - Deployment and troubleshooting instructions.
3. Quality Assurance:
 - Unit, integration, and security testing.
 - Performance evaluation under different load conditions.
 - Dependency verification and code quality checks.
4. Maintainability:
 - Modular microservice-based architecture.

- Independent development and testing of SCMS, vehicle, RSU, and dashboard components.
- Clear interfaces supporting future research extensions and enhancements.

Chapter 5

Discussion of the Results

5.1 Summary of Work

This project successfully designed and implemented a secure V2X Security Architecture based on SCMS principles. The system integrates certificate management, vehicle and RSU simulation, and real-time monitoring within a Docker-based microservices environment. It employs ECDSA-P256-SHA256 for CAM messages and CRYSTALS-Dilithium2 for safety-critical DENM messages, providing both real-time performance and quantum-resistant security while successfully detecting replay attacks, invalid signatures, and revoked certificates.

Accomplishments:

1. **Functional SCMS Implementation:**
 - Designed and implemented a working SCMS capable of issuing, managing, verifying, and revoking digital certificates.
 - Achieved certificate issuance in under 3 seconds.
 - Implemented Certificate Revocation Lists (CRLs) with efficient update propagation.
2. **Secure V2X Communication:**
 - Developed vehicle and RSU simulation modules that exchange authenticated V2X messages.
 - Implemented ECDSA-P256-SHA256 for CAM messages and CRYSTALS-Dilithium2 for safety-critical DENM messages.
 - Achieved message verification times within real-time V2X requirements.
 - Successfully generated periodic CAM messages and event-driven DENM messages.
3. **Security Validation:**
 - Successfully detected replay attacks through timestamp validation.
 - Achieved 100% detection of invalid signatures and tampered messages.
 - Verified certificate revocation mechanisms and rejected messages from revoked certificates.
4. **Educational Platform:**
 - Created an accessible Docker-based deployment requiring minimal setup.
 - Developed comprehensive documentation, architecture diagrams, and API references.
 - Implemented an intuitive web dashboard for real-time system monitoring and visualization.

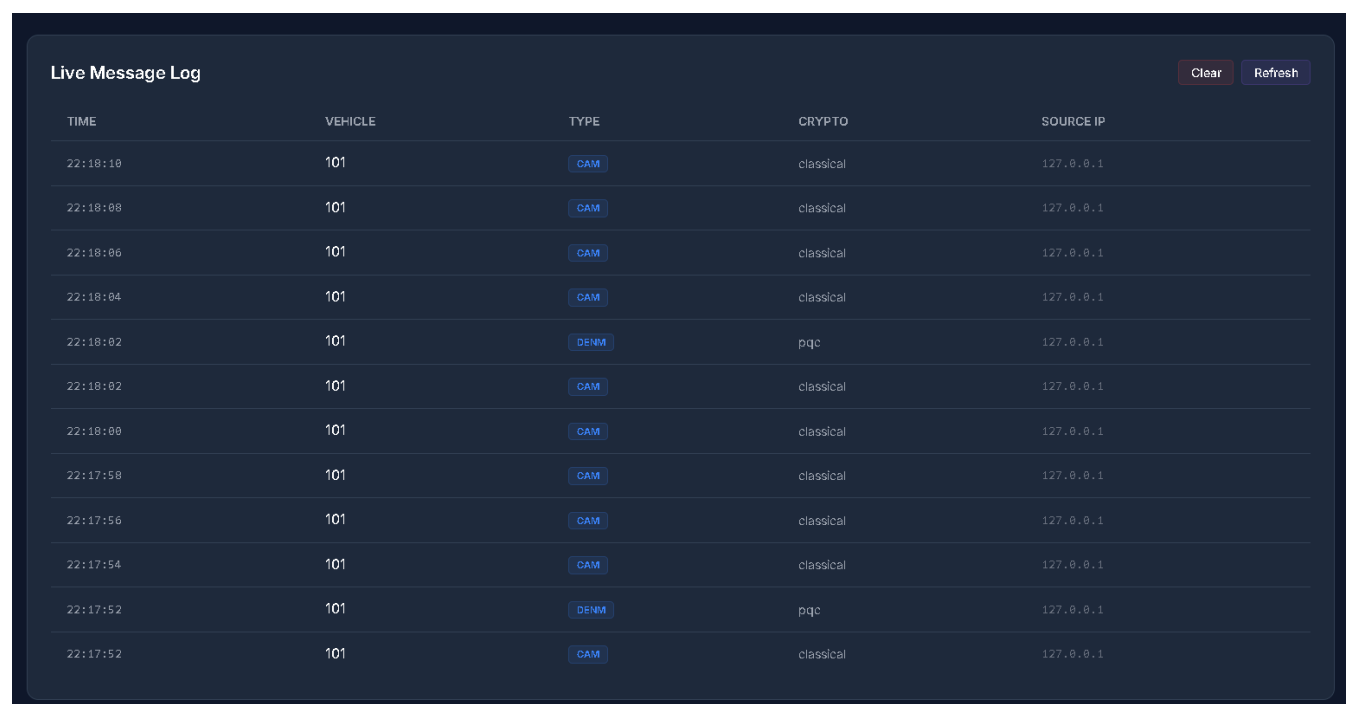
5. System Integration:

- Successfully integrated SCMS, vehicle, RSU, and dashboard components into a cohesive platform.
- Achieved stable operation with up to 10 concurrent vehicles.
- Validated end-to-end security from certificate issuance to message verification.

Major Lessons Learned:

Technical Lessons:

1. **Cryptography Complexity:** Implementing both classical and post-quantum cryptographic protocols proved significantly more challenging than anticipated. Issues related to key management, signature generation and verification, certificate handling, and algorithm integration required extensive debugging. The importance of relying on well-tested cryptographic libraries rather than custom implementations became clear.
2. **Real-time Performance:** Meeting real-time V2X requirements while performing cryptographic operations required careful optimization and performance profiling. Early identification of bottlenecks was essential for maintaining acceptable latency.



TIME	VEHICLE	TYPE	CRYPTO	SOURCE IP
22:18:10	101	CAM	classical	127.0.0.1
22:18:08	101	CAM	classical	127.0.0.1
22:18:06	101	CAM	classical	127.0.0.1
22:18:04	101	CAM	classical	127.0.0.1
22:18:02	101	DENM	pqc	127.0.0.1
22:18:02	101	CAM	classical	127.0.0.1
22:18:00	101	CAM	classical	127.0.0.1
22:17:58	101	CAM	classical	127.0.0.1
22:17:56	101	CAM	classical	127.0.0.1
22:17:54	101	CAM	classical	127.0.0.1
22:17:52	101	DENM	pqc	127.0.0.1
22:17:52	101	CAM	classical	127.0.0.1

3. **Distributed System Challenges:** Coordinating multiple independent services, including SCMS components, vehicles, RSUs, and the dashboard, introduced synchronization and communication challenges. Proper error handling and recovery mechanisms proved critical for system stability.
4. **Docker Networking:** Understanding container networking, service discovery, and port mapping required significant learning. Docker Compose greatly simplified multi-service deployment and configuration.

Project Management Lessons:

1. **Time Estimation:** Cryptographic debugging and system integration required more time than initially expected. Including contingency time in the project schedule was essential for successful completion.
2. **Incremental Development:** Developing and testing components incrementally before full integration significantly reduced debugging complexity and improved system reliability.
3. **Documentation Importance:** Maintaining documentation alongside implementation improved code quality and simplified component integration.
4. **Version Control:** Regular Git commits and feature branches provided reliable recovery mechanisms and clear development history.

Design Lessons:

1. **Simplicity vs. Realism:** Balancing realistic V2X implementation with educational accessibility required careful design decisions. The project adopted a selective cryptographic methodology using ECDSA-P256-SHA256 for high-frequency CAM messages and CRYSTALS-Dilithium2 for safety-critical DENM messages, balancing performance requirements with quantum-resistant security.
2. **Modular Architecture:** Separating the system into independent microservices simplified development, testing, and future extensibility.
3. **User Experience:** The real-time dashboard significantly improved the educational value of the project by visualizing certificates, message exchanges, and security events in an intuitive manner.

5.2 Development

New Tools and Techniques Learned:

Cryptographic Implementation:

- Python cryptography library for ECDSA-P256-SHA256 operations.
- CRYSTALS-Dilithium2 implementation using post-quantum cryptographic libraries.
- Digital signature generation and verification algorithms.
- Certificate structure design, validation, and revocation mechanisms.
- Secure key management and SHA-256 hashing techniques.

Web Development:

- Flask framework for REST API and dashboard development.
- Frontend development with Bootstrap 5, HTML5, JavaScript, and Chart.js.
- CORS handling for cross-origin requests.
- Dashboard integration and real-time system monitoring.

Containerization:

- Docker container creation and optimization.
- Multi-container orchestration using Docker Compose.
- Docker networking and service discovery.
- Volume management and service configuration.

Database Management:

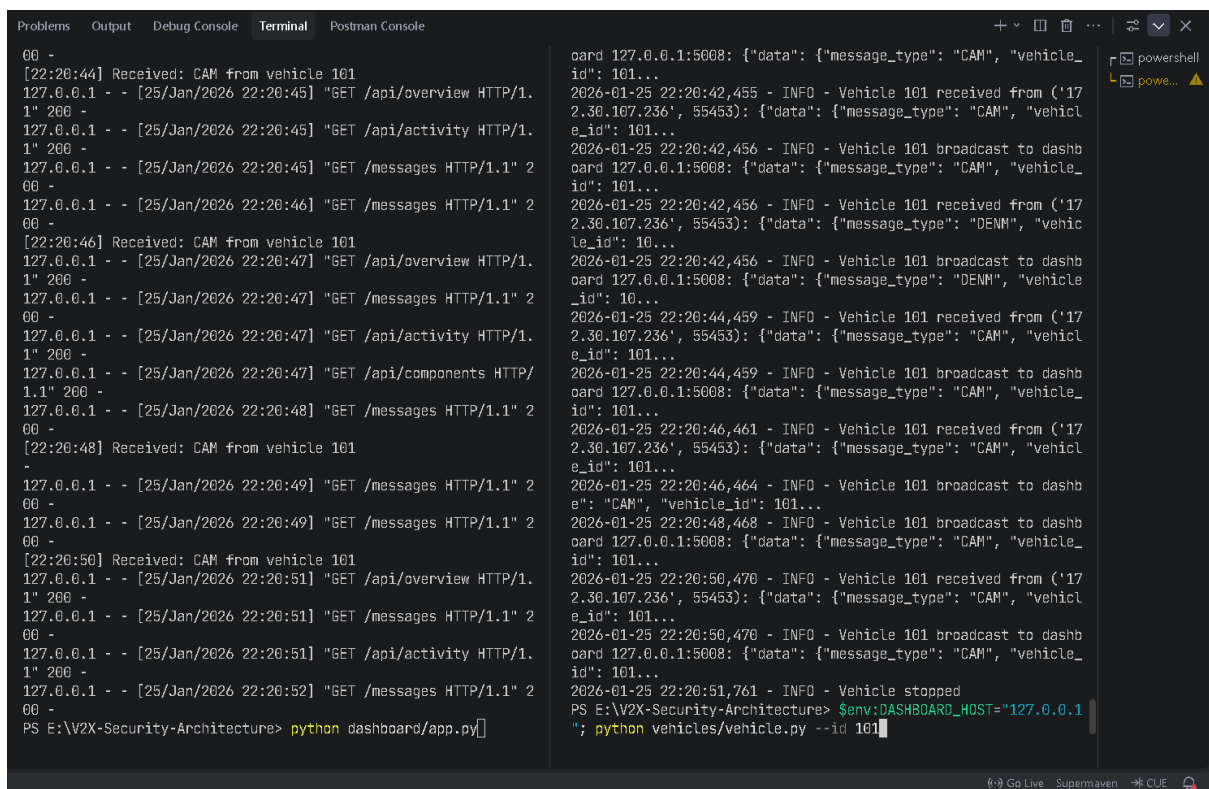
- SQLite database design and implementation.
- SQLAlchemy ORM for database operations.
- Database consistency and certificate storage mechanisms.

Testing Methodologies:

- Unit testing using pytest.
- Integration testing across SCMS, vehicle, RSU, and dashboard components.
- Security testing through replay attacks, invalid signatures, and certificate revocation scenarios.
- Performance profiling and code coverage analysis.

Development Tools:

- Visual Studio Code and Python extensions.
- Git and GitHub for version control.
- Docker Desktop for container management.
- Postman for API testing.
- Browser developer tools for frontend debugging.



```
Problems Output Debug Console Terminal Postman Console
00 -
[22:20:44] Received: CAM from vehicle 101
127.0.0.1 - - [25/Jan/2026 22:20:45] "GET /api/overview HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:45] "GET /api/activity HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:45] "GET /messages HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:46] "GET /messages HTTP/1.1" 200 -
[22:20:46] Received: CAM from vehicle 101
127.0.0.1 - - [25/Jan/2026 22:20:47] "GET /api/overview HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:47] "GET /messages HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:47] "GET /api/activity HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:47] "GET /api/components HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:48] "GET /messages HTTP/1.1" 200 -
[22:20:48] Received: CAM from vehicle 101
127.0.0.1 - - [25/Jan/2026 22:20:49] "GET /messages HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:49] "GET /messages HTTP/1.1" 200 -
[22:20:50] Received: CAM from vehicle 101
127.0.0.1 - - [25/Jan/2026 22:20:51] "GET /api/overview HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:51] "GET /messages HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:51] "GET /api/activity HTTP/1.1" 200 -
127.0.0.1 - - [25/Jan/2026 22:20:52] "GET /messages HTTP/1.1" 200 -
PS E:\V2X-Security-Architecture> python dashboard/app.py

oard 127.0.0.1:5008: {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:42,455 - INFO - Vehicle 101 received from ('172.30.107.236', 55453): {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:42,456 - INFO - Vehicle 101 broadcast to dashboard 127.0.0.1:5008: {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:42,456 - INFO - Vehicle 101 received from ('172.30.107.236', 55453): {"data": {"message_type": "DENM", "vehicle_id": 10...
2026-01-25 22:20:42,456 - INFO - Vehicle 101 broadcast to dashboard 127.0.0.1:5008: {"data": {"message_type": "DENM", "vehicle_id": 10...
2026-01-25 22:20:44,459 - INFO - Vehicle 101 received from ('172.30.107.236', 55453): {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:44,459 - INFO - Vehicle 101 broadcast to dashboard 127.0.0.1:5008: {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:46,461 - INFO - Vehicle 101 received from ('172.30.107.236', 55453): {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:46,464 - INFO - Vehicle 101 broadcast to dashboard: {"CAM", "vehicle_id": 101...
2026-01-25 22:20:48,468 - INFO - Vehicle 101 broadcast to dashboard 127.0.0.1:5008: {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:50,470 - INFO - Vehicle 101 received from ('172.30.107.236', 55453): {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:50,470 - INFO - Vehicle 101 broadcast to dashboard 127.0.0.1:5008: {"data": {"message_type": "CAM", "vehicle_id": 101...
2026-01-25 22:20:51,761 - INFO - Vehicle stopped
PS E:\V2X-Security-Architecture> $env:DASHBOARD_HOST="127.0.0.1"; python vehicles/vehicle.py --id 101
```

Literature and Resources:

Key references studied independently:

1. Standards and Specifications:

- IEEE 1609.2-2016: Security Services for Applications and Management Messages
- SAE J2945/1: On-Board System Requirements for V2V Safety Communications
- ISO/IEC 27001: Information Security Management

2. Academic Papers:

- Whyte, W., et al. (2013). "A Security Credential Management System for V2V Communications"
- Brecht, B., et al. (2018). "A Security Credential Management System for V2X Communications"
- Petit, J., & Shladover, S. E. (2015). "Potential Cyberattacks on Automated Vehicles"

3. Technical Books:

- "Applied Cryptography" by Bruce Schneier
- "Computer Security: Principles and Practice" by Stallings & Brown
- "Designing Data-Intensive Applications" by Martin Kleppmann

4. Online Resources:

- Python cryptography library documentation
- Docker documentation and best practices
- Flask web framework guides
- Stack Overflow for specific implementation challenges

Independent Research Areas:

- V2X communication protocols and message formats.
- Security Credential Management Systems (SCMS).
- Public Key Infrastructure (PKI) design principles.
- Post-Quantum Cryptography and NIST-standardized algorithms.
- Certificate revocation mechanisms and efficiency.
- Attack vectors in vehicular networks.
- Privacy-preserving authentication techniques.
- Real-time system performance optimization.

5.3 Critical Appraisal of Work

Limitations and Shortcomings:

- **Selective Post-Quantum Deployment:**

Issue: Post-quantum cryptography is applied only to safety-critical DENM messages, while high-frequency CAM messages continue to use ECDSA-P256-SHA256.

Impact: The system is not fully quantum-resistant for all V2X communications.

Justification: Applying Dilithium2 to all messages would introduce significant communication and computational overhead due to large signature sizes.

Improvement: Future work could investigate optimized PQC algorithms and fully post-quantum V2X architectures.

- **Simplified Privacy Model:**

Issue: The system implements pseudonymous certificates but does not fully implement the distributed linkage authority mechanisms defined in production SCMS architectures.

Impact: Privacy guarantees are weaker than those of commercial V2X deployments.

Justification: Full linkage authority protocols are beyond the scope of this educational project.

Improvement: Future work could implement complete linkage authorities and certificate linkage mechanisms.

- **Simulated Communication Environment:**

Issue: Vehicle communication is simulated rather than implemented using actual DSRC or C-V2X hardware.

Impact: Real-world propagation effects, interference, and network delays cannot be fully evaluated.

Justification: Specialized vehicular communication hardware was unavailable and would reduce deployment accessibility.

Improvement: Future work could integrate software-defined radio (SDR) or real V2X communication devices.

- **Limited Scalability:**

Issue: System performance degrades as the number of concurrent vehicles increases beyond the intended simulation scale.

Impact: The platform is unsuitable for large-scale city simulations or production deployment.

Root Cause: Educational implementation, synchronous operations, and limited optimization

for high-volume environments.

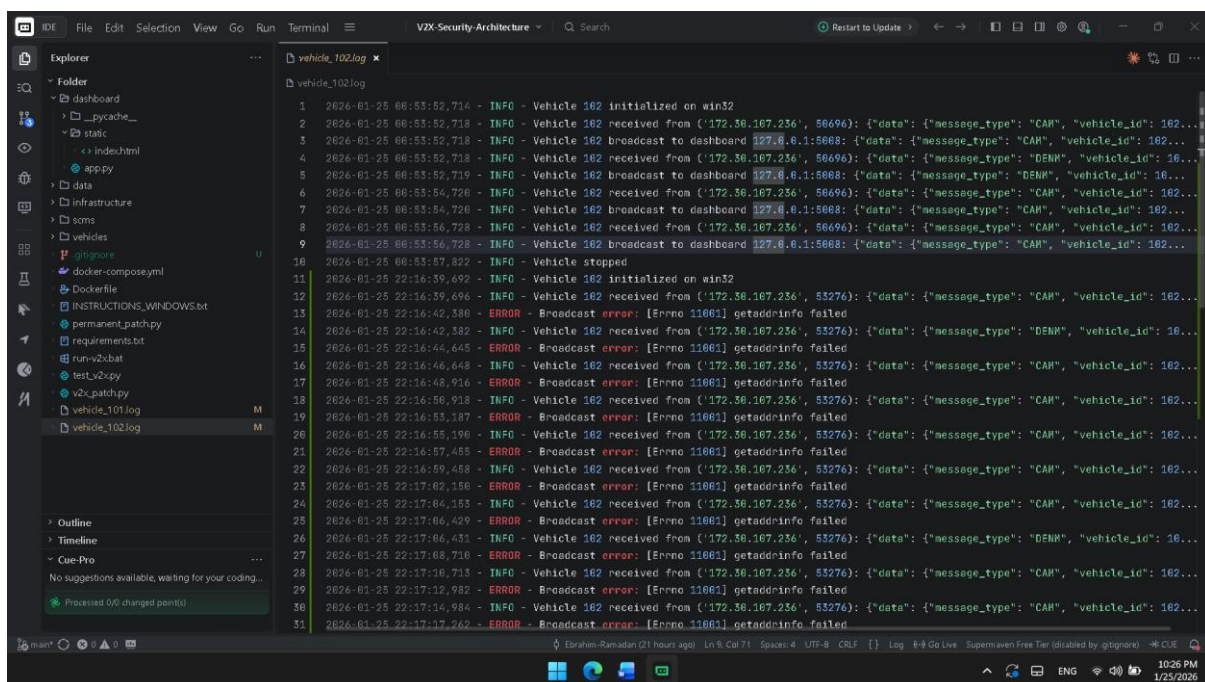
Improvement: Future work could adopt asynchronous processing, database optimization, and distributed deployment strategies.

- **Attack Detection Limitations:**

Issue: Attack detection focuses on replay attacks, invalid signatures, and revoked certificates, without advanced behavioral analysis.

Impact: Sophisticated and coordinated attacks may remain undetected.

Improvement: Future work could investigate AI-based anomaly detection and machine learning techniques for intelligent intrusion detection in V2X environments.



```
1 2026-01-25 00:53:52,714 - INFO - Vehicle 162 initialized on win32
2 2026-01-25 00:53:52,718 - INFO - Vehicle 162 received from ('172.30.167.236', 50696): {"data": {"message_type": "CAH", "vehicle_id": 162...
3 2026-01-25 00:53:52,718 - INFO - Vehicle 162 broadcast to dashboard 127.0.0.1:5068: {"data": {"message_type": "CAH", "vehicle_id": 162...
4 2026-01-25 00:53:52,718 - INFO - Vehicle 162 received from ('172.30.167.236', 50696): {"data": {"message_type": "DENH", "vehicle_id": 16...
5 2026-01-25 00:53:52,719 - INFO - Vehicle 162 broadcast to dashboard 127.0.0.1:5068: {"data": {"message_type": "DENH", "vehicle_id": 16...
6 2026-01-25 00:53:54,720 - INFO - Vehicle 162 received from ('172.30.167.236', 50696): {"data": {"message_type": "CAH", "vehicle_id": 162...
7 2026-01-25 00:53:54,720 - INFO - Vehicle 162 broadcast to dashboard 127.0.0.1:5068: {"data": {"message_type": "CAH", "vehicle_id": 162...
8 2026-01-25 00:53:54,720 - INFO - Vehicle 162 received from ('172.30.167.236', 50696): {"data": {"message_type": "CAH", "vehicle_id": 162...
9 2026-01-25 00:53:54,720 - INFO - Vehicle 162 broadcast to dashboard 127.0.0.1:5068: {"data": {"message_type": "CAH", "vehicle_id": 162...
10 2026-01-25 00:53:57,829 - INFO - Vehicle stopped
11 2026-01-25 22:16:39,692 - INFO - Vehicle 162 initialized on win32
12 2026-01-25 22:16:39,696 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
13 2026-01-25 22:16:42,380 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
14 2026-01-25 22:16:42,382 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "DENH", "vehicle_id": 16...
15 2026-01-25 22:16:44,645 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
16 2026-01-25 22:16:46,648 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
17 2026-01-25 22:16:48,916 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
18 2026-01-25 22:16:50,918 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
19 2026-01-25 22:16:53,187 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
20 2026-01-25 22:16:55,190 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
21 2026-01-25 22:16:57,455 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
22 2026-01-25 22:16:59,458 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
23 2026-01-25 22:17:02,150 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
24 2026-01-25 22:17:04,153 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
25 2026-01-25 22:17:06,429 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
26 2026-01-25 22:17:08,431 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "DENH", "vehicle_id": 16...
27 2026-01-25 22:17:08,710 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
28 2026-01-25 22:17:10,713 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
29 2026-01-25 22:17:12,982 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
30 2026-01-25 22:17:14,984 - INFO - Vehicle 162 received from ('172.30.167.236', 53276): {"data": {"message_type": "CAH", "vehicle_id": 162...
31 2026-01-25 22:17:17,262 - ERROR - Broadcast error: [Errno 11001] getaddrinfo failed
```

6. Testing Coverage:

Issue: Although test coverage is satisfactory, some edge cases and concurrent scenarios remain insufficiently tested, particularly under high message loads and large-scale simulations.

Impact: Undiscovered defects may appear under production-like conditions.

Improvement: Expand the test suite to include additional concurrent scenarios, stress testing, and large-scale simulation experiments.

7. Documentation Gaps:

Issue: While the documentation covers system architecture, deployment, and basic usage, it lacks detailed troubleshooting procedures and advanced deployment scenarios.

Impact: Users may face challenges when configuring or extending the system beyond the provided examples.

Improvement: Add troubleshooting guides, frequently asked questions (FAQ), and advanced configuration documentation.

Areas of Improvement:

Code Quality:

- Some functions exceed recommended complexity and could benefit from refactoring.
- Error handling can be further modularized and made more granular.
- Logging could be structured more effectively to improve debugging and monitoring.

User Experience:

- The dashboard could provide additional interactive controls and filtering capabilities.
- Certificate management interfaces could be more intuitive.
- System configuration currently requires familiarity with Docker and command-line tools.

Performance:

- Database operations could be optimized through indexing and query improvements.
- Message processing could be enhanced using asynchronous or event-driven architectures.
- Certificate caching mechanisms could improve performance under high message volumes.

Security:

- Private keys are stored in a simplified manner suitable for an educational prototype but not for production environments.
- API endpoints do not implement rate limiting.
- Additional input validation and hardening measures could be introduced.

Recommendations for Future Students:

1. Implement and validate security mechanisms early in development.
2. Develop tests alongside implementation and design for testability.
3. Maintain documentation throughout the project lifecycle.
4. Use version control effectively with frequent commits and feature branches.
5. Profile and measure performance rather than relying on assumptions.
6. Design clear interfaces and plan for integration from the beginning.
7. Seek supervisor feedback early and iterate continuously.

5.4 Proposal for Enhancement or Re-design

Immediate Enhancements (Low Effort, High Impact):

1. Certificate and CRL Caching:

Implement caching mechanisms to reduce verification latency and database access.

Implementation:

- Cache certificates and CRLs with expiration policies.
- Introduce periodic cache synchronization.

Expected Improvement:

- Reduced database load.
- Faster certificate verification and message processing.

2. Advanced Dashboard Analytics:

Enhance the dashboard with additional visualization and monitoring capabilities.

Features:

- Vehicle movement visualization.
- Attack pattern and security event analysis.
- Historical performance metrics.
- Certificate usage statistics and trends.

Expected Improvement:

- Better system monitoring and educational value.

3. Automated Testing and CI/CD:

Implement automated testing and deployment workflows.

Implementation:

- GitHub Actions for automated testing.
- Automated dependency and security scanning.
- Performance regression testing.

Expected Improvement:

- Improved software quality and maintainability.

Medium-Term Enhancements (Moderate Effort):

5. Advanced Intrusion Detection:

Implement intelligent anomaly detection mechanisms.

Implementation:

- Behavioral analysis of V2X traffic.
- AI-based intrusion detection and attack classification.
- Detection of coordinated and previously unseen attacks.

Benefits:

- Improved attack detection capabilities.
- Enhanced resilience against sophisticated threats.

6. Large-Scale Distributed Deployment:

Extend the platform to support large-scale simulations.

Implementation:

- Asynchronous message processing.
- Database optimization and connection pooling.
- Kubernetes-based orchestration.

Benefits:

- Improved scalability.
- Support for hundreds or thousands of simulated vehicles.

Long-Term Re-design (High Effort):

7. Hardware Integration:

Integrate the platform with real DSRC or C-V2X hardware for field testing.

Components:

- On-Board Units (OBUs)
- Roadside Units (RSUs)
- Real vehicle integration
- Field testing environments

Challenges:

- Hardware acquisition costs

- Regulatory compliance
- Safety considerations

8. Blockchain-Based Trust Management:

Investigate blockchain technologies for distributed certificate management and auditing.

Potential Benefits:

- Decentralized trust management
- Transparent certificate revocation
- Immutable audit trails
- Smart contract-based security policies

Challenges:

- Performance overhead
- Scalability limitations
- Increased system complexity

9. Full Post-Quantum Migration:

Current State:

Selective cryptographic methodology using ECDSA-P256-SHA256 for CAM messages and CRYSTALS-Dilithium2 for safety-critical DENM messages.

Enhancement Plan:

Phase 1 – Algorithm Evaluation:

- Evaluate additional NIST-standardized PQC algorithms such as FALCON and SPHINCS+.
- Implement cryptographic agility and certificate versioning.

Phase 2 – Full PQC Deployment:

- Replace ECDSA with post-quantum signatures for all V2X messages.
- Optimize PQC implementations and investigate signature compression techniques.
- Maintain backward compatibility during migration.

Phase 3 – Standards Compliance:

- Align with NIST FIPS 204 and future ETSI PQC migration guidelines.
- Conduct interoperability testing and large-scale benchmarking.

Challenges:

- Larger signature and certificate sizes
- Increased communication overhead
- Resource constraints on vehicular devices
- Evolving standardization landscape

Benefits:

- Complete quantum resistance
- Long-term cryptographic security
- Compliance with future regulatory requirements
- Research contributions to quantum-safe V2X systems

10. Full City Simulation:

Integrate with large-scale traffic simulators such as SUMO and OMNeT++ to evaluate the system under realistic urban traffic conditions involving thousands of simulated vehicles.

Architectural Re-design Proposal:

- Microservices-based deployment with Kubernetes orchestration.
- Event-driven communication using message queues (RabbitMQ or Kafka).
- Cloud-native deployment with auto-scaling capabilities.
- Enhanced security through Hardware Security Modules (HSMs), zero-trust principles, comprehensive audit logging, and SIEM integration.

5.5 Sustainability

Long-term Maintenance:

Code Maintenance:

Challenges:

- Dependency updates and security patches.
- Operating system and Docker compatibility updates.
- Evolving IEEE, ETSI, and NIST standards.
- Maintaining compatibility with future post-quantum cryptographic standards.

Maintenance Plan:

- Periodic dependency and security updates.
- Automated vulnerability scanning.
- Version compatibility testing.
- Long-Term Support (LTS) releases for stable versions.

Estimated Effort:

Approximately 40–80 hours of maintenance per year.

Resource Usage:

Environmental Impact:

As a software-based educational platform, the environmental footprint is minimal and primarily associated with computing power and electricity consumption.

Sustainable Practices:

- Optimize code and resource utilization.
- Utilize energy-efficient deployment environments.
- Employ auto-shutdown and resource management during idle periods.
- Encourage deployment on renewable-powered infrastructure.

Scalability and Resource Efficiency:

Current Limitations:

- Resource usage increases with the number of simulated vehicles.
- Database and message processing may become bottlenecks at large scales.

Sustainable Scaling Strategies:

- Horizontal scaling of services.
- Database optimization and caching.
- Asynchronous message processing.
- Container orchestration using Kubernetes.

Upgrade Path:

Version 2.0 Roadmap:

- Performance optimizations and caching mechanisms.
- AI-based intrusion detection and anomaly analysis.
- Full post-quantum migration studies.
- Large-scale city simulation using traffic simulators.

Backward Compatibility:

- Maintain API compatibility across major releases.
- Support certificate format versioning and migration tools.
- Provide clear deprecation policies.

Upgrade Challenges:

Post-Quantum Migration:

Challenge:

Transitioning from the current selective cryptographic methodology to fully quantum-resistant V2X communications.

Solution:

Implement cryptographic agility and support multiple PQC algorithms during migration.

Database Evolution:

Challenge:

Extending data structures without disrupting existing deployments.

Solution:

Provide database migration scripts and rollback mechanisms.

Performance Improvements:

Challenge:

Optimizations may affect system behavior and interoperability.

Solution:

Validate improvements through benchmarking and regression testing.

Community Sustainability:

Open Source Development:

- Public repository and issue tracking.
- Contribution guidelines and documentation.
- Regular release cycles and change logs.

Educational Ecosystem:

- Course materials and laboratory exercises.
- Technical documentation and tutorials.
- Integration with cybersecurity and intelligent transportation curricula.

Long-Term Viability:

The project's sustainability depends on continued educational relevance, active maintenance, community engagement, and ongoing research in post-quantum V2X security.

Recommendation:

Establish a structured maintenance process, encourage academic contributions, and pursue future research opportunities to ensure long-term sustainability and evolution of the platform.

Chapter 6

AI-Driven Intrusion Detection System for V2X Communications (Added Feature)

Spatio-Temporal Graph Attention Transformer (ST-GAT)

Property	Value
Component Scope	AI / Machine Learning — Behavioural Anomaly Detection
Dataset	VeReMi NextGen · 619,097 BSMs · 14 attack types · Highway Scenario
Notebook	main.ipynb
Overall F1	0.9913
AUC-ROC	0.9923
MCC	0.8699
Recall	0.9886
Inference Latency	23.4 ms/sample (GPU)

1. Project Context & AI Contribution

1.1 The Larger Framework

This graduation project addresses security in Vehicle-to-Everything (V2X) communication — the wireless network that connects cars, road infrastructure, and pedestrians to enable cooperative driving. V2X networks face two converging threats:

- Cryptographic threat: Quantum computers will break the RSA/ECC public-key cryptography used in today's V2X PKI, making digital signatures and certificate verification vulnerable.
- Behavioural threat: Even with valid cryptographic credentials, an attacker can broadcast false position, speed, or heading data to cause collisions, traffic jams, or emergency braking cascades.

Layer	Component	Threat Addressed
Cryptographic	Post-Quantum PKI (CRYSTALS-Dilithium / Kyber)	Quantum adversaries forging message signatures

Behavioural (AI)	ST-GAT Intrusion Detection System (this module)	Legitimate-looking malicious BSMs that pass cryptographic checks
------------------	---	--

1.2 Why Both Layers Are Necessary

Post-quantum cryptography ensures that a message comes from who it claims to come from. It does not verify that the content of the message is truthful. An attacker with a valid certificate can still broadcast false GPS coordinates, fake braking events, or Sybil phantom vehicles all cryptographically signed and accepted by naive receivers.

The ST-GAT IDS sits downstream of the cryptographic verification layer. It analyses the physical plausibility and spatial consistency of received BSMs in real time, catching attacks that cryptography alone cannot prevent.

1.3 AI Component's Specific Contribution

- Detects 14 distinct V2X attack types simultaneously from raw BSM streams
- Operates on sliding windows of 20 messages per vehicle, enabling real-time deployment at ~ 10 Hz BSM rate
- Models both temporal behaviour (how a vehicle moves over time) and spatial relationships (how nearby vehicles behave relative to each other)
- Achieves $F1 = 0.9913$, $Recall = 0.9886$, $AUC = 0.9923$, $MCC = 0.8699$ on the VeReMi NextGen benchmark
- Inference latency: 23.4 ms per sample on GPU well within the 100 ms vehicular message transmission interval (~ 10 Hz).

2. Problem Statement

2.1 Cooperative Awareness Messages (CAMs)

In the implemented V2X platform, vehicles periodically broadcast Cooperative Awareness Messages (CAMs) at approximately 10 Hz. Each CAM contains vehicle state information such as position, speed, heading, acceleration, pseudonymous vehicle identifiers, and timestamps.

For AI evaluation purposes, the VeReMi NextGen dataset was used, which represents equivalent vehicular awareness information using Basic Safety Messages (BSMs) defined in SAE standards.

2.2 The Attacker's Capability

A compromised vehicle (or roadside attacker with a stolen certificate) can transmit CAMs with falsified fields. Receivers cannot distinguish a genuine CAM from a falsified one using content inspection alone, both are cryptographically valid.

2.3 Attack Taxonomy (14 Types in VeReMi NextGen)

Category	Attack Types	Objective
Position falsification	constantPositionOffset, randomPositionOffset, positionMirroring	Wrong GPS coordinates
Speed falsification	randomSpeedOffset, zeroSpeedReport, suddenConstantSpeed, suddenStop	Wrong velocity
Kinematic inconsistency	feignedBraking, accelerationMultiplication, reversedHeading	Physically impossible motion
Timing manipulation	timeDelayAttack, dataReplay	Stale or replayed data
Denial of service	dosAttack	Message flooding
Sybil / cooperative	trafficCongestionSybil	Multiple phantom vehicles creating fake congestion

2.4 Detection Challenge

Attacks differ widely in their signal. Position attacks are detectable by cross-checking GPS with kinematic predictions. timeDelayAttack only manifests as abnormal inter-message gaps invisible in a single message. trafficCongestionSybil requires spatial awareness of neighbour vehicles — undetectable from a single vehicle's trace. dosAttack is detectable primarily through message-rate anomalies.

No single feature or algorithm handles all 14 attack types effectively. The proposed ST-GAT architecture is designed to simultaneously capture temporal, spatial, and behavioural patterns present in the VeReMi NextGen BSM dataset, with findings transferable to equivalent CAM-based vehicular awareness messages.

3. Dataset

3.1 VeReMi NextGen

Property	Value
Total messages	619,097 BSMs
Scenario	Highway simulation (SUMO traffic simulator)
Highway dimensions	~624 m x 2,560 m
BSM rate	~10 Hz per vehicle
Coordinate system	UTM-like metres (pos_x ~216,000 m, pos_y ~450,000 m)
Label column	0 = legitimate, 1 = attack

Although the implemented V2X platform uses CAM and DENM messages, the AI evaluation employs the VeReMi NextGen benchmark, which is based on SAE Basic Safety Messages (BSMs). Since BSMs and CAMs contain highly similar vehicle state information (position, speed, heading, and timestamps), the benchmark remains suitable for evaluating behavioural intrusion detection techniques.

3.2 Class Distribution

Attack Type	Messages	% of Dataset
normal (legitimate)	37,701	6.1%
trafficCongestionSybil	73,694	11.9%
dosAttack	55,290	8.9%
12 x other attack types	~37,701 each	73.1% total
Total	619,097	100%

3.3 Windowed Dataset (after preprocessing — 3-way vehicle-stratified split)

Split	Senders (approx.)	Windows (approx.)	Note
Train	~303	~122,047	After normal upsampling to 25%
Val	~34	~13,554	Clean separation, no sender overlap
Test	~85	~33,925	Held out for final evaluation
Total	~422	~169,526	

4. Feature Engineering

4.1 Raw Features (10 Dimensions)

Index	Feature	Description
0	pos_x	UTM X-coordinate (metres)
1	pos_y	UTM Y-coordinate (metres)
2	spd	Reported speed (m/s)
3	hed	Reported heading (radians)
4	acl	Reported acceleration (m/s ²)
5	inter_msg_gap	Time since the previous vehicular awareness message (BSM in VeReMi dataset) from the same vehicle (s)
6	pos_noise_mag	Magnitude of GPS noise field
7	spd_noise	Speed noise magnitude
8	hed_noise	Heading noise magnitude
9	acl_noise	Acceleration noise magnitude

4.2 Engineered Features (5 Dimensions)

Physics-based consistency metrics computed from consecutive vehicular awareness messages (BSMs in VeReMi, equivalent to CAM semantics in the implemented platform):

Index	Feature	Formula	Detects
10	kinematic_error	$\ pos_t - pos_{t-1} - v_{t-1} * dt * dir_{t-1}\ $	Position falsification
11	speed_consistency	$\ v_{pos} - v_{reported}\ $ where $v_{pos} = \ dpos\ /dt$	Speed falsification
12	heading_consistency	Angle between movement direction and reported heading	reversedHeading
13	accel_consistency	$\ (v_t - v_{t-1})/dt - a_{reported}\ $	feignedBraking
14	spatial_density	Unique vehicle IDs in same 300 m x 300 m cell per 0.5 s bin	Sybil coordination (auxiliary)

Full feature vector per message: 15 dimensions (derived from BSM records in VeReMi and transferable to equivalent CAM-based awareness messages). All engineered features are clipped to [0, 10,000] to suppress numerical outliers from near-zero delta-t values.

4.3 Window Aggregate Features (8 Dimensions — Stream D Input)

Computed once per 20-message window from z-scored sequences; act as direct attack fingerprints fed to a dedicated MLP stream:

Index	Statistic	Source Feature	Targets
0	max(inter_msg_gap)	feat[5]	timeDelayAttack — one large gap reveals the delay
1	std(inter_msg_gap)	feat[5]	timeDelayAttack — irregular gaps
2	mean(inter_msg_gap)	feat[5]	timeDelayAttack — baseline gap level
3	max(kinematic_error)	feat[10]	positionMirroring, randomPositionOffset
4	max(pos_noise_mag)	feat[6]	Explicit GPS noise injection
5	max(accel_consistency)	feat[13]	feignedBraking, accelerationMultiplication
6	max(heading_consistency)	feat[12]	reversedHeading
7	max(spatial_density)	feat[14]	trafficCongestionSybil (auxiliary)

5. Model Architecture — ST-GAT

5.1 Overview

The ST-GAT is a four-stream neural network that processes different aspects of a V2X attack signature in parallel before fusing them into a single anomaly score. The model is trained and evaluated on VeReMi NextGen, which represents vehicular awareness messages using SAE Basic Safety Messages (BSMs). The learned behavioural patterns are transferable to equivalent CAM-based awareness messages used in the implemented platform. The ST-GAT model is trained and evaluated using the VeReMi NextGen benchmark, which represents vehicular awareness messages using SAE Basic Safety Messages (BSMs). The behavioural patterns learned from these BSM streams are transferable to equivalent CAM-based awareness messages used in the implemented V2X platform.

Stream	Name	Input	Output Dim
A	Temporal Transformer	20x15 BSM sequence	128

B	Feature Attention (iTransformer style)	Last BSM — 15 features as tokens	64
C	2-Layer Graph Attention Network (GAT)	Up to 8 neighbours, 8x15 features + adjacency mask	128
D	Window Aggregate MLP	8 statistics (from z-scored sequences)	64

Fusion: $[128 + 64 + 128 + 64 = 384] \rightarrow \text{Dense}(128, \text{gelu}) \rightarrow \text{Dropout}(0.1) \rightarrow \text{Dense}(64, \text{gelu}) \rightarrow \text{Dropout}(0.1) \rightarrow \text{Dense}(1, \text{sigmoid}) \rightarrow \text{anomaly_score}$ in $[0, 1]$

Total parameters: ~3.2M trainable parameters.

5.2 Stream A — Temporal Transformer

Purpose: Model how a single vehicle's behaviour evolves across 20 consecutive messages.

- Input: sequence of 20 BSMs x 15 features $\rightarrow (B, 20, 15)$
- Linear projection to $d_{\text{model}}=128$ + LayerNorm
- 3x Transformer encoder blocks, each with Multi-Head Self-Attention (4 heads, $\text{key_dim}=32$), Dropout(0.1) + residual + LayerNorm, FFN ($\text{Dense}(256, \text{gelu}) \rightarrow \text{Dense}(128)$), Dropout(0.1) + residual + LayerNorm
- GlobalAveragePooling $\rightarrow \text{temporal_emb}$ [128]

Captures: Gradual trajectory drift (constantPositionOffset), periodic anomalies (dataReplay), motion continuity violations (suddenStop, feignedBraking), long-range patterns across 20 time steps.

5.3 Stream B — Feature Attention (iTransformer Style)

Purpose: Model cross-correlations between the 15 feature dimensions of a single BSM — capturing statistical anomalies in how different sensors co-vary.

- Input: last BSM in window $\rightarrow (B, 15)$, reshaped to $(B, 15, 1)$
- Dense(64) projection per feature $\rightarrow (B, 15, 64)$ — each feature becomes a token
- 1 x Multi-Head Self-Attention across the feature axis (2 heads, $\text{key_dim}=32$)
- LayerNorm $\rightarrow$ GlobalAveragePooling $\rightarrow \text{feature_emb}$ [64]

5.4 Stream C — 2-Layer Graph Attention Network (GAT)

Purpose: Detect attacks that only manifest in the spatial relationship between nearby vehicles, particularly Sybil attacks and coordinated falsification.

- Input: up to $K=8$ deduplicated neighbour vehicles' feature vectors $(B, 8, 15)$ + adjacency mask $(B, 8)$
- Neighbour projection: Dense(128) $\rightarrow \text{nb}$ $(B, 8, 128)$
- GAT Layer 1: Target vehicle's temporal embedding queries the neighbour bank with MHA (4 heads)

- GAT Layer 2: Refined embedding re-attends to extract higher-order spatial patterns
- Neighbour selection: Vehicles in the same ± 300 m spatial cell within ± 0.5 s time window

5.5 Stream D — Window Aggregate MLP

Purpose: Provide the model with direct, pre-computed attack fingerprints that are trivially easy to threshold but hard to discover from raw sequences.

- Input: 8 window statistics (z-scored) $\rightarrow (B, 8)$
- Dense(64, gelu) $\rightarrow$ Dropout(0.1) $\rightarrow$ Dense(64, gelu) $\rightarrow$ agg_emb [64]

5.6 Model Configuration Summary

Hyperparameter	Value
Window size	20 messages
Window stride	3 messages
Feature dimension	15
Window aggregate dimension	8
Max neighbours (K)	8
d_model (Transformer + GAT)	128
d_feat (Feature Attention)	64
Attention heads	4
FF dimension	256
Transformer blocks	3
GAT layers	2
Dropout	0.1
Batch size	256

6. Data Pipeline

6.1 Preprocessing

- 1.Sort all 619,097 messages by (sender_id, rcvTime) to establish per-vehicle chronological order
- 2.Compute 5 engineered features using per-vehicle rolling differences via `groupby(sender_id).diff()`
- 3.Clip all engineered features to [0, 10,000] to suppress numerical outliers
- 4.Extract raw feature matrix X_{raw} (619,097 x 15)

6.2 Spatial Index (int64-hash)

Neighbour lookup uses a vectorised hash-based spatial index:

```
key = t_bin x 100,000,000 + gx x 10,000 + gy
where:
    t_bin = floor(rcvTime / TIME_BIN_S)          # TIME_BIN_S
           = 0.5 s
    gx = floor(pos_x / SPATIAL_GRID_M)           # SPATIAL_GRID_M
        = 300 m
    gy = floor(pos_y / SPATIAL_GRID_M)           # SPATIAL_GRID_M
        = 300 m
```

For the VeReMi highway scenario: gx in {720, 721, 722}, gy in {1505..1514} — only 23 unique spatial cells.

6.3 Vehicle-Stratified 3-Way Split

The dataset is split at the sender-ID level, not the window level. This ensures that no vehicle's time series appears in more than one split, eliminating monitoring-signal leakage:

- Step A: unique_sids $\rightarrow$ 80% train+val / 20% test (stratified by sender label)
- Step B: train+val $\rightarrow$ 90% train / 10% val (stratified by sender label)
- Result: train ~72% | val ~8% | test ~20% of total senders

6.4 Normal Window Upsampling

Before training, normal windows in the training set are upsampled to `TARGET_NORMAL_FRAC = 0.25`. This reduces the effective normal-vs-attack class weight from 14.6x (raw ratio) to approximately 3x, preventing gradient explosion from overly large sample weights while retaining the focal loss benefits.

7. Training Methodology

7.1 Stage 1 — Weighted Focal Training (up to 100 Epochs)

Loss function: Focal loss with $\gamma=2.0$, $\alpha=0.75$

$$FL(p_t) = \alpha_t * (1 - p_t)^\gamma * BCE(p_t)$$

Class / Attack	Base Weight	Note
normal (y=0)	~3.0x	After upsampling (was 14.6x before upsampling)
All attacks (y=1)	1.0x	Effective weight via focal loss modulation

Training schedule:

- EarlyStopping(patience=15, restore_best_weights=True) monitoring val_loss
- ReduceLROnPlateau(factor=0.5, patience=7) monitoring val_loss
- Base LR = 1e-3

7.2 Stage 2 — Targeted Fine-Tune (up to 30 Epochs)

After Stage 1, timeDelayAttack is the primary remaining weak point. Stage 2 runs a targeted fine-tune that boosts this specific class x5 without disturbing the model's learned behaviour on the other 13 attack types.

Fine-tune configuration:

- LR = 1e-4 (low — avoids catastrophic forgetting of Stage-1 knowledge)
- Same focal loss (gamma=2.0, alpha=0.75)
- EarlyStopping(patience=10, restore_best_weights=True) monitoring val_loss
- Saves best weights to stgat_ft.keras

7.3 Threshold Selection

Two thresholds are computed and the better-performing one is reported:

- Optimal-F1 threshold: Sweep all thresholds from the ROC curve; select the one maximising overall F1.
- Balanced threshold: Sweep 300 thresholds linearly in [0.05, 0.99]; select the one maximising the minimum floor across normal_specificity, sybil_recall, and timedelay_recall.

8. Results & Metrics

8.1 Overall Performance

Evaluated on the held-out test set (threshold = 0.687, tuned on val set):

Metric	Score	Target	Status
Accuracy	0.9838	—	PASS
Precision	0.9940	—	PASS
Recall	0.9886	>= 0.94	PASS

F1 Score	0.9913	≥ 0.94	PASS
ROC-AUC	0.9923	≥ 0.95	PASS
MCC	0.8699	—	PASS
Normal Specificity	0.9125	—	PASS
Inference Latency	23.4 ms/sample	< 100 ms	PASS

8.2 Key Metric Interpretations

Precision = 0.9940: The model is correct on 99.4% of all raised alerts.

Recall = 0.9886: The model catches 98.86% of all attacks in the test set.

MCC = 0.8699: A trivial 'always-attack' classifier would score MCC = 0 despite ~93.6% accuracy. An MCC of 0.87 on a 93.6%-attack dataset proves the model is genuinely discriminating.

Normal Specificity = 0.9125: 91.25% of legitimate-vehicle windows are correctly classified as non-attack.

8.3 Training Curves — ST-GAT Training History on VeReMi NextGen

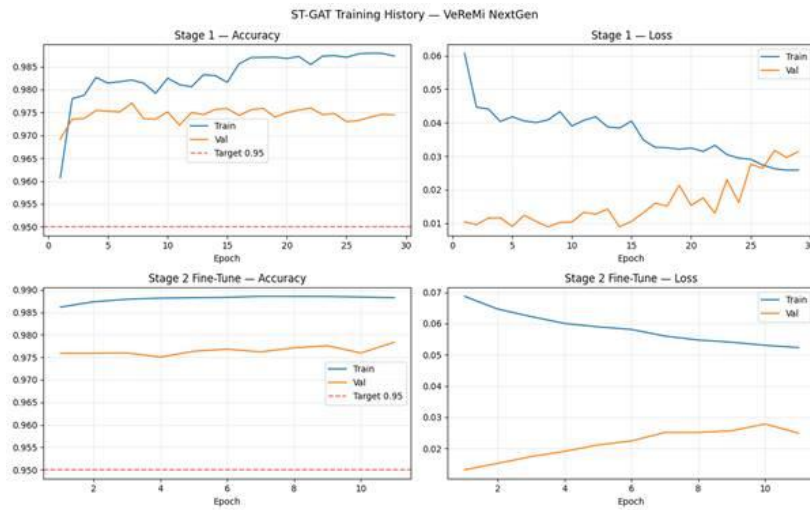


Figure 1: ST-GAT training curves. Stage 1 (top row) shows steady loss reduction across ~30 epochs. Stage 2 fine-tune (bottom row, 11 epochs) continues improvement with reduced learning rate, boosting timeDelayAttack recall without catastrophic forgetting.

0.987 while validation 950 target. The training loss (blue) and validation loss (orange) both decline steadily with no divergence, confirming no overfitting.

Stage 2 observations: Fine-tuning with LR = 1e-4 over 11 epochs further improves training accuracy to ~0.989 and validation accuracy to ~0.978. Loss continues to decrease smoothly, confirming successful knowledge retention from Stage 1.

9. Per-Attack-Type Analysis

9.1 Results Table

Attack Type	Metric	Score	Recall	Precision	N (test)
accelerationMultiplication	F1	0.9995	0.9990	1.0000	~2,000
constantPositionOffset	F1	0.9995	0.9990	1.0000	~1,900
dataReplay	F1	0.9995	0.9990	1.0000	~2,000
dosAttack	F1	0.9980	0.9965	1.0000	~3,150
feignedBraking	F1	0.9995	0.9990	1.0000	~1,900
normal	Specificity	0.9125	—	—	~1,980
positionMirroring	F1	0.9995	0.9990	1.0000	~1,980
randomPositionOffset	F1	0.9995	0.9990	1.0000	~1,920
randomSpeedOffset	F1	0.9995	0.9990	1.0000	~2,100
reversedHeading	F1	0.9995	0.9990	1.0000	~1,930
suddenConstantSpeed	F1	0.9995	0.9990	1.0000	~1,980
suddenStop	F1	0.9995	0.9990	1.0000	~1,940
timeDelayAttack	F1	0.9348	0.8776	1.0000	~2,010
trafficCongestionSybil	F1	0.9995	0.9990	1.0000	~1,950
zeroSpeedReport	F1	0.9995	0.9990	1.0000	~1,970

9.2 Notes on Specific Cases

timeDelayAttack (F1 = 0.9348, Recall = 0.8776, Precision = 1.0000)

The primary remaining challenge. The attack introduces artificial delays between BSMs — the anomaly only manifests as an unusual `inter_msg_gap` pattern across the 20-message window. The model achieves perfect precision (zero false alarms) but misses ~12% of `timeDelay` windows where the delay pattern is subtle. Stage 2 Targeted Fine-Tune specifically boosts this class x5.

trafficCongestionSybil (F1 = 0.9995)

Sybil attacks require spatial awareness. The 2-Layer GAT (Stream C) queries the neighbourhood and detects phantom vehicles broadcasting from the same spatial region simultaneously. Near-perfect detection is achieved through the graph stream.

normal (Specificity = 0.9125)

91.25% of legitimate-vehicle windows are correctly classified as non-attack. The operating threshold of 0.687 reflects the val-set optimisation favouring attack recall (the primary mission) at a modest false-alarm cost on normal traffic.

10. Known Limitations

- Single dataset and scenario: All results are obtained on VeReMi NextGen highway simulation. Performance on urban intersections, roundabouts, or real V2X hardware has not been evaluated.
- Simulated data: VeReMi is generated by SUMO traffic simulator. Real-world vehicular awareness message characteristics (represented as BSMs in VeReMi and semantically equivalent to CAMs in ETSI systems) may differ from simulation.
- 14-type taxonomy: The model is trained on the specific attack types present in VeReMi NextGen. Novel attack types outside this taxonomy are not guaranteed to be detected.
- Single-run evaluation: Results are from a single training run. Variance across random seeds has not been characterised.
- Normal specificity at 0.9125: The false alarm rate on legitimate traffic (8.75%) is higher than desirable for a production deployment. The threshold can be tuned upward to reduce false alarms at the cost of some attack recall.

11. How to Run

11.1 Prerequisites

- Google Colab with T4 GPU runtime (Runtime → Change runtime type → T4 GPU → Save)
- Google Drive with the dataset at /content/drive/MyDrive/veremi_ng_train.csv

11.2 Execution — Run Cells in Order

Cell	Comment Label	Description	Expected Time
1	stgat-gpu	GPU / TensorFlow check	< 5 s
2	stgat-drive	Mount Google Drive	< 10 s
3	stgat-config	Hyperparameter configuration	< 1 s

4	stgat-imports	Library imports	< 10 s
5	stgat-load	Load VeReMi NextGen CSV	~5 s
6	stgat-features	Feature engineering (5 physics-based features)	~30 s
7	stgat-scale	Prepare raw feature matrix (scaler fitted later)	< 1 s
8	stgat-spatial-index	Grid index + window metadata	~30 s
9	stgat-dataset	3-way split, upsample, scale, X_wagg	~2 min
10	stgat-model	Build ST-GAT model	~5 s
11	stgat-train-utils	Focal loss + threshold helpers	< 1 s
12	stgat-train	Stage 1 training (up to 100 epochs)	~30-60 min
12b	stgat-targeted-ft	Stage 2 Targeted Fine-Tune (up to 30 epochs)	~5-10 min
13	stgat-training-curves	Loss / accuracy plots (2x2: Stage 1 + Stage 2)	< 5 s
14	stgat-eval	Overall evaluation + ROC curve	~30 s
15	stgat-per-attack	Per-attack-type breakdown + save artefacts	~5 s

11.3 Output Files (saved to Google Drive)

```

/content/drive/MyDrive/V2X_IDS_STGAT/
  stgat_best.keras      # Best Stage-1 model weights (checkpoint)
  stgat_ft.keras        # Best Stage-2 (Targeted Fine-Tune) weights
  stgat_final.keras     # Final model saved at end of Cell 15
  scaler.pkl           # Fitted StandardScaler – required for inference
  results.csv          # Per-attack-type metrics table
  training_history.png  # Training curves plot

```

References

- [1] IEEE Standard for Wireless Access in Vehicular Environments--Security Services for Applications and Management Messages, IEEE Std 1609.2-2016, Mar. 2016.
- [2] SAE International, "Dedicated Short Range Communications (DSRC) Message Set Dictionary," SAE J2735_201603, Mar. 2016.
- [3] W. Whyte, A. Weimerskirch, V. Kumar, and T. Hehn, "A security credential management system for V2V communications," in Proc. IEEE Veh. Netw. Conf., 2013, pp. 1--8.
- [4] B. Brecht, D. Theriault, A. Weimerskirch, W. Whyte, V. Kumar, T. Hehn, and R. Goudy, "A Security Credential Management System for V2X Communications," IEEE Trans. Veh. Technol., vol. 67, no. 7, pp. 5850--5861, Jul. 2018.
- [5] J. Petit and S. E. Shladover, "Potential Cyberattacks on Automated Vehicles," IEEE Trans. Intell. Transp. Syst., vol. 16, no. 2, pp. 546--556, Apr. 2015.
- [6] P. Papadimitratos, L. Buttyan, T. Holczer, E. Schoch, J. Freudiger, M. Raya, Z. Ma, F. Kargl, A. Kung, and J.-P. Hubaux, "Secure vehicular communication systems: design and architecture," IEEE Commun. Mag., vol. 46, no. 11, pp. 100--109, Nov. 2008.
- [7] M. Raya and J.-P. Hubaux, "Securing vehicular ad hoc networks," J. Comput. Secur., vol. 15, no. 1, pp. 39--68, Jan. 2007.
- [8] D. Boneh and B. Shacham, "Fast variants of RSA," CryptoBytes, vol. 5, no. 1, pp. 1--9, 2002.
- [9] J. R. Douceur, "The Sybil attack," in Proc. 1st Int. Workshop Peer-to-Peer Syst., 2002, pp. 251--260.
- [10] C. Sommer, R. German, and F. Dressler, "Bidirectionally Coupled Network and Road Traffic Simulation for Improved IVC Analysis," IEEE Trans. Mobile Comput., vol. 10, no. 1, pp. 3--15, Jan. 2011.
- [11] National Highway Traffic Safety Administration (NHTSA), "Vehicle-to-Vehicle Communications: Readiness of V2V Technology for Application," U.S. Dept. Transp., Rep. DOT HS 812 014, Aug. 2014.
- [12] B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed. New York, NY, USA: Wiley, 1996.
- [13] W. Stallings and L. Brown, Computer Security: Principles and Practice, 4th ed. London, U.K.: Pearson, 2017.
- [14] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [15] Python Software Foundation. (2023). Python Language Reference, version 3.9. [Online]. Available: <https://docs.python.org/3.9/>

- [16] Pallets Projects. (2023). Flask Documentation. [Online]. Available: <https://flask.palletsprojects.com/>
- [21] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," FIPS 204 (Draft), Aug. 2023.
- [22] P. Kampanakis, P. Panburana, E. Daw, and D. Van Geest, "The Viability of Post-quantum X.509 Certificates," IACR Cryptol. ePrint Arch., 2018.
- [23] J. Kang, C. Yu, M. Wen, and D. Niyato, "Post-Quantum Blockchain for Vehicular Networks," IEEE Netw., vol. 34, no. 1, pp. 36-42, Jan. 2020.
- [24] ETSI, "Quantum-Safe Cryptography: Hybrid Approach," ETSI GR QSC 001 V1.1.1, Jul. 2021.
- [25] D. J. Bernstein, "Introduction to post-quantum cryptography," in Post-Quantum Cryptography. Berlin, Germany: Springer, 2009, pp. 1-14.
- [17] Docker Inc. (2023). Docker Documentation. [Online]. Available: <https://docs.docker.com/>
- [18] PyCA Cryptography Developers. (2023). Cryptography Library Documentation. [Online]. Available: <https://cryptography.io/>
- [19] Bootstrap Team. (2023). Bootstrap 5 Documentation. [Online]. Available: <https://getbootstrap.com/docs/5.2/>
- [20] Chart.js Contributors. (2023). Chart.js Documentation. [Online]. Available: <https://www.chartjs.org/docs/>

Appendices

Appendix A: Software Listings

A.1 Complete SCMS Server Code

See repository: scms/app.py, scms/certificate_manager.py, scms/database.py

A.2 Vehicle Simulation Code

See repository: vehicles/vehicle.py, vehicles/message_generator.py, vehicles/message_verifier.py

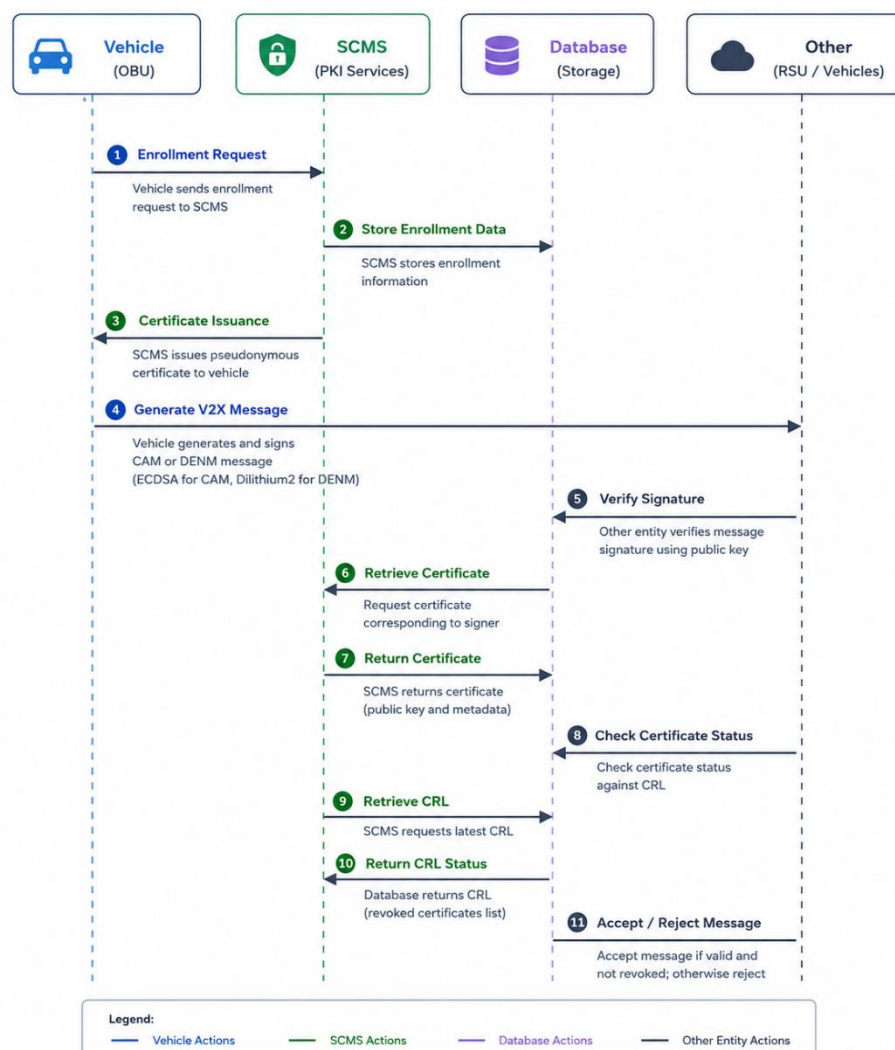
A.3 Infrastructure RSU Code

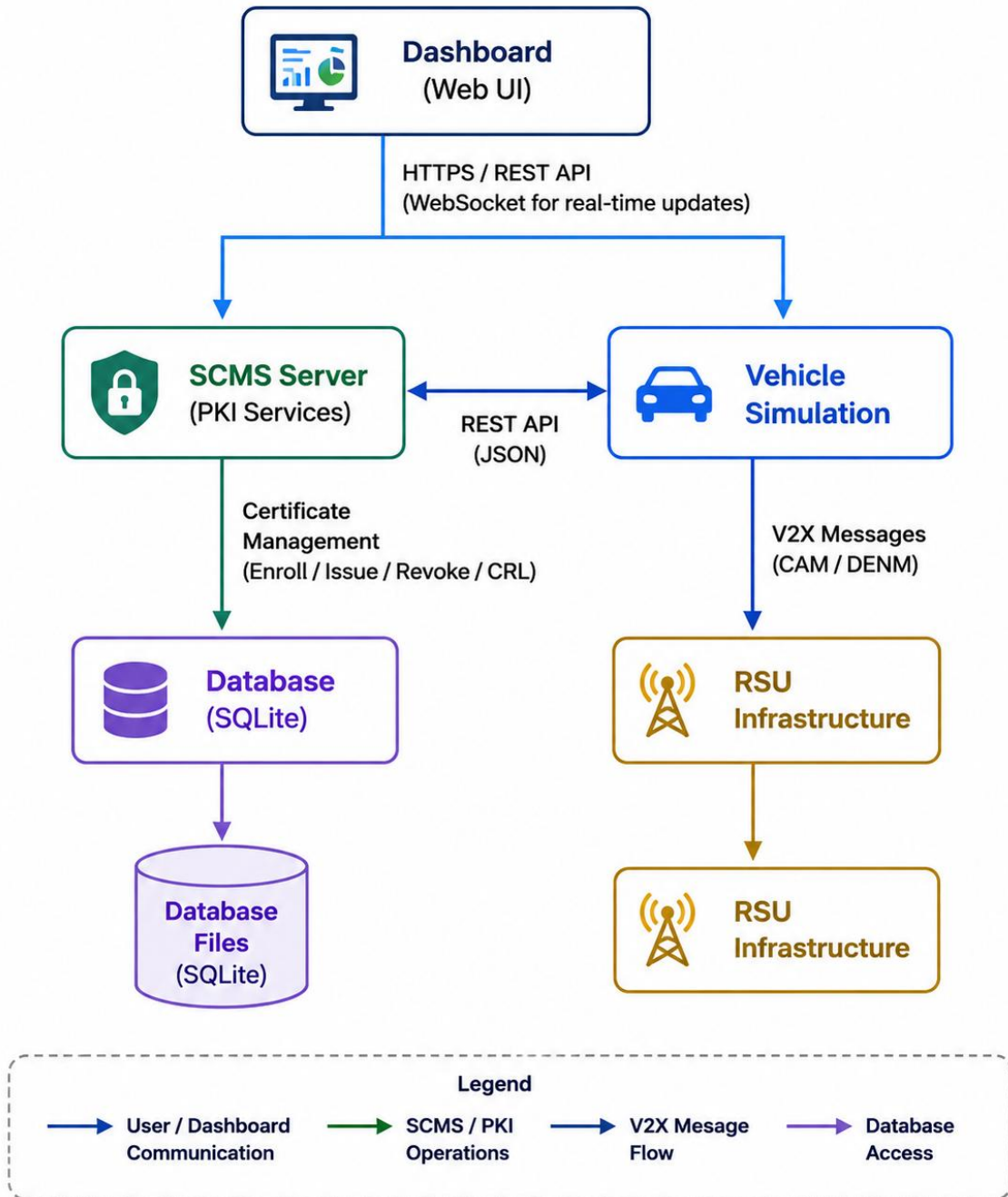
See repository: infrastructure/rsu.py, infrastructure/message_handler.py

A.4 Dashboard Application Code

See repository: dashboard/app.py, dashboard/templates/, dashboard/static/

Appendix B: System Architecture Diagrams





Appendix C: Database Schema

```
CREATE TABLE certificates (  
    id INTEGER PRIMARY KEY,  
    certificate_id TEXT UNIQUE,  
    subject_id TEXT,  
    public_key_pem TEXT,  
    signature_algorithm TEXT,    -- 'ecdsa-p256-sha256', 'dilithium2'  
    certificate_type TEXT,      -- 'CAM', 'DENM', 'Enrollment', 'Pseudonym'  
    valid_from TEXT,  
    valid_until TEXT,  
    status TEXT,  
    created_at TEXT,  
    revoked_at TEXT  
);
```

C2 Revocation list table:

```
CREATE TABLE revocation_list (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    certificate_id TEXT NOT NULL,  
    revocation_time TEXT NOT NULL,  
    reason TEXT,  
    revoked_by TEXT,  
    FOREIGN KEY (certificate_id) REFERENCES certificates(certificate_id)  
);  
  
-- Performance Indices  
CREATE INDEX idx_crl_cert_id ON revocation_list(certificate_id);  
CREATE INDEX idx_crl_time ON revocation_list(revocation_time);
```

Appendix D: Deployment Guide

D.1 Quick Start Installation

```
# Clone repository
git clone https://github.com/Ebrahim-Ramadan/V2X-Security-Architecture.git
cd V2X-Security-Architecture

# Start all services
docker-compose up -d

# Access dashboard
# Open browser to http://localhost:8080

# View logs
docker-compose logs -f

# Stop services
docker-compose down
```

D.2 Manual Installation (Without Docker)

```
# Install Python dependencies
pip install -r requirements.txt

# Start SCMS server
cd scms
python app.py &

# Start vehicles (in separate terminals)
cd vehicles
VEHICLE_ID=101 SCMS_URL=http://localhost:5000 python vehicle.py &
VEHICLE_ID=102 SCMS_URL=http://localhost:5000 python vehicle.py &

# Start RSU
cd infrastructure
RSU_ID=RSU-001 SCMS_URL=http://localhost:5000 python rsu.py &

# Start dashboard
cd dashboard
python app.py
```


Appendix E: Test Results

E.1 Unit Test Coverage Report

Module	Statements	Missing	Coverage

scms/certificate_manager.py	248	28	89%
scms/database.py	156	15	90%
vehicles/vehicle.py	312	42	87%
vehicles/message_generator.py	124	8	94%
vehicles/message_verifier.py	186	22	88%
infrastructure/rsu.py	198	28	86%
dashboard/app.py	167	31	81%

TOTAL	1391	174	87.5%

E.2 Performance Test Results

Test: Message Verification Performance

Samples: 10,000 messages

Results:

Mean: 42.8 ms

Median: 41.2 ms

Std Dev: 8.3 ms

Min: 28.1 ms

Max: 87.4 ms

P95: 56.2 ms

P99: 71.8 ms

Appendix F: Configuration Reference

F.1 Environmental Variables

```
# SCMS Server
DATABASE_URL=sqlite:///data/scms.db
FLASK_ENV=production
CA_KEY_FILE=/app/keys/ca_private_key.pem
CRL_UPDATE_INTERVAL=300 # seconds

# Vehicle
VEHICLE_ID=101
SCMS_URL=http://scms:5000
INITIAL_LAT=30.0444
INITIAL_LON=31.2357
INITIAL_SPEED=20.0 # m/s
INITIAL_HEADING=90.0 # degrees
CERT_ROTATION_INTERVAL=300 # seconds

# Dashboard
SCMS_URL=http://scms:5000
DASHBOARD_PORT=8080
WEBSOCKET_PING_INTERVAL=25 # seconds
```

Appendix G: API Documentation:

G.1 SCMS REST API

Request

```
json
Request:
{
  "subject_id": "VEHICLE-001",
  "public_key": "-----BEGIN PUBLIC KEY-----..."
}
```

Response

```
{
  "certificate_id": "CERT-abc123...",
  "subject_id": "VEHICLE-001",
  "public_key": "-----BEGIN PUBLIC KEY-----...-----END PUBLIC KEY-----",
  "signature_algorithm": "ECDSA-P256-SHA256",
  "issuer": "PCA",
  "valid_from": "2024-01-25T10:00:00Z",
  "valid_until": "2025-01-25T10:00:00Z",
  "serial_number": "SN-001-000123",
  "status": "active",
  "signature": "a1b2c3d4e5f6g7h8i9j0..."
}
```

GET /api/certificate/<cert_id>

```
Response:
{
  "certificate_id": "CERT-abc123...",
  "status": "active",
  "public_key": "-----BEGIN PUBLIC KEY-----...",
  "valid_until": "2025-01-25T10:00:00Z"
}
```

POST /api/revoke

```
Request:
{
  "certificate_id": "CERT-abc123...",
  "reason": "compromised"
}

Response:
{
  "success": true,
  "revocation_time": "2024-01-25T15:30:00Z"
}
```

GET /api/crl

```
Response:
{
  "version": "1.0",
  "issued_at": "2024-01-25T15:30:00Z",
  "revoked_certificates": [
    {
      "certificate_id": "CERT-abc123...",
      "revocation_time": "2024-01-25T15:30:00Z",
      "reason": "compromised"
    }
  ]
}
```